

Package ‘curvana’

June 8, 2026

Type Package

Title Tools to Analyze AFM Force–Distance Curves

Version 0.1.0

Description An all-in-one R package to read, manage, and analyze atomic force microscopy (AFM) force–distance curves, with support for parallel processing and user-supplied metadata.

License MIT + file LICENSE

Encoding UTF-8

URL <https://github.com/GaryZhangYue/fdafmR>

BugReports <https://github.com/GaryZhangYue/fdafmR/issues>

Imports bs4Dash,
 circlize,
 ComplexHeatmap,
 cowplot,
 future,
 future.apply,
 dplyr,
 ggplot2,
 ggrepel,
 ggpubr,
 methods,
 readxl,
 rhandsontable,
 signal,
 shiny,
 tibble,
 writexl,
 zip

RoxygenNote 7.3.3

Suggests DT,
 knitr,
 rmarkdown

VignetteBuilder knitr

Contents

analyze_a_curve_adhesive_force	2
analyze_a_curve_area	4
analyze_a_curve_interaction_distance	6
analyze_a_curve_noise	9
analyze_baseline	11
analyze_curves_adhesive_force	13
analyze_curves_all_analytical_metrics	14
analyze_curves_energy	17
analyze_curves_interaction_distance	18
analyze_curves_noise	20
analyze_sensitivity	21
area_trapezoid	22
area_triangle	23
calc_sensitivity	23
combineFdObj	25
createFdObjFromFolder	25
createFdObjFromJPKFolder	27
crossing_x0	29
denoise_a_curve	29
denoise_curves	31
extract	32
fdObj-class	32
find_baseline	33
plot_all_curve_metrics	35
plot_a_curve_metrics	37
plot_complex_heatmap	39
plot_deflection_curves	40
plot_deflection_curves_by_index	41
plot_fd_curves	43
plot_metric_violin	44
plot_pca_biplot	46
plot_raw_deflection_heatmap	47
run_curvana_app	49
show,FdObj-method	49
transform_a_curve	50
transform_curves	52
Index	55

analyze_a_curve_adhesive_force

Adhesive Force for a Single Transformed AFM Curve

Description

Extracts the adhesion minimum from one transformed force-separation curve by locating the most negative value in `force_nN` and reporting its magnitude together with the corresponding `separation_distance_nm`. The function first validates that the input is a data frame containing the required transformed-curve columns, coerces both columns to numeric, and discards rows with non-finite values before searching for the minimum force.

Usage

```
analyze_a_curve_adhesive_force(curve_df)
```

Arguments

curve_df A data.frame with columns:

- **separation_distance_nm** Numeric tip-sample separation (nm).
- **force_nN** Numeric force (nN).

Details

For valid transformed curves, adhesion is defined here as a negative force excursion. If the minimum force is below zero, its sign is flipped so the reported `adhesive_force_nN` is a positive adhesion magnitude. If the curve contains no negative force values, the function records this as "no detectable adhesion" and returns `adhesive_force_nN = 0` with `separation_distance_nm = NA_real_`. If the input cannot be analyzed at all because required columns are missing or no finite x-y pairs remain after cleaning, both returned values are `NA`.

Value

A named numeric vector of length 2:

adhesive_force_nN The negated most negative force value in the curve. Returns `0` when the curve is valid but contains no negative force values, and `NA_real_` when the curve cannot be analyzed.

separation_distance_nm The separation distance (nm) at which the adhesive-force minimum occurs. Returns `NA_real_` when no adhesion is detected or when the input is invalid.

This makes the output distinguish between a valid "no adhesion" case (`0/NA`) and a failed analysis case (`NA/NA`).

Examples

```
transformed_df <- data.frame(
  separation_distance_nm = c(
    0.22196440, 0.19896440, 0.17496440, 0.23529773, 0.21129773,
    0.18829773, 0.16429773, 0.14129773, 0.11729773, 0.17763107,
    0.15463107, 0.13063107, 0.19096440, 0.08363107, -0.02270227,
    -0.21336893, -0.31970227, -0.42603560, -0.53336893, -0.47303560,
    -0.49703560, -0.43670227, -0.54403560, -0.65036893, -0.67336893,
    -0.78070227, -0.72036893, -0.49436893, -0.18403560, 0.04196440,
    0.18563107, 0.49496440, 1.13863107, 3.03229773, 5.09163107,
    7.15196440, 8.21129773, 9.27163107, 10.58096440, 11.97463107,
    13.28396440, 14.42763107, 15.48796440, 16.38063107, 17.44096440,
    18.41696440, 19.39396440, 20.28663107, 21.26363107, 22.24063107,
    23.13329773, 24.11029773, 25.08629773, 26.06329773, 27.12263107,
    28.01629773, 28.99329773, 29.96929773, 30.94629773, 31.92229773,
    32.89929773, 33.87529773, 34.85229773, 35.82829773, 36.72196440,
    37.69896440, 38.75829773, 39.73529773, 40.79463107, 41.77163107,
    42.74763107, 43.72463107, 44.70063107, 45.67763107, 46.57129773,
    47.54729773, 48.52429773, 49.58363107, 50.47729773, 51.45329773,
    52.43029773, 53.49063107, 54.38329773, 55.44363107, 56.33629773,
    57.31329773, 58.28929773, 59.34963107, 60.32663107, 61.21929773,
    62.19629773, 63.25563107, 64.23263107, 65.37529773, 66.26896440,
    67.24496440, 68.13863107, 69.03229773, 70.00829773, 70.90196440
```

```

),
force_nN = c(
  2.588500000, 2.488500000, 2.388500000, 2.296833333, 2.196833333,
  2.096833333, 1.996833333, 1.896833333, 1.796833333, 1.705166667,
  1.605166667, 1.505166667, 1.413500000, 1.305166667, 1.196833333,
  1.080166667, 0.971833333, 0.863500000, 0.755166667, 0.663500000,
  0.563500000, 0.471833333, 0.363500000, 0.255166667, 0.155166667,
  0.046833333, -0.044833333, -0.119833333, -0.186500000, -0.261500000,
  -0.344833333, -0.411500000, -0.444833333, -0.353166667, -0.244833333,
  -0.136500000, -0.128166667, -0.119833333, -0.086500000, -0.044833333,
  -0.011500000, 0.005166667, 0.013500000, 0.005166667, 0.013500000,
  0.013500000, 0.013500000, 0.005166667, 0.005166667, 0.005166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, 0.005166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.003166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.011500000,
  -0.011500000, -0.003166667, -0.003166667, 0.005166667, 0.005166667,
  0.005166667, 0.005166667, 0.005166667, -0.003166667,
  -0.003166667, -0.003166667, 0.005166667, -0.003166667, -0.003166667,
  -0.003166667, 0.005166667, -0.003166667, 0.005166667, -0.003166667,
  -0.003166667, -0.003166667, 0.005166667, 0.005166667, -0.003166667,
  -0.003166667, 0.005166667, 0.005166667, 0.021833333, 0.013500000,
  0.013500000, 0.005166667, -0.003166667, -0.003166667, -0.011500000
)
)
analyze_a_curve_adhesive_force(transformed_df)

```

analyze_a_curve_area	Analyze the adhesive and repulsive areas of a single AFM curve relative to noise-band thresholds
----------------------	--

Description

Calculates the total area of a transformed force-separation curve that lies outside a user-defined noise band. Positive force contributions above noiseBand_high are reported as *repulsive_area*, whereas negative force contributions below noiseBand_low are reported as *adhesive_area*. The function expects one curve in tabular form with separation_distance_nm as the x-axis and force_nN as the y-axis, and it returns the two areas separately rather than combining them into a signed net integral.

Usage

```

analyze_a_curve_area(
  curve_df,
  noiseBand_low = -.Machine$double.eps,
  noiseBand_high = .Machine$double.eps
)

```

Arguments

curve_df	A data frame with at least two columns: separation_distance_nm Numeric x-coordinates (nm). Values < 0 are clamped to 0.
----------	--

	force_nN	Numeric y-coordinates (nN).
noiseBand_low		Numeric lower noise-band threshold (default: <code>-.Machine\$double.eps</code>). Adhesive area is counted where <code>force_nN < noiseBand_low</code> .
noiseBand_high		Numeric upper noise-band threshold (default: <code>.Machine\$double.eps</code>). Repulsive area is counted where <code>force_nN > noiseBand_high</code> .

Details

Integration is performed in the original row order, so the function respects the incoming trace geometry and does not sort or otherwise reorder points. For each adjacent pair of points, the function checks whether the segment lies fully above the upper threshold, fully below the lower threshold, entirely inside the noise band, or crosses one or both thresholds. Fully above-threshold and fully below-threshold segments are integrated as trapezoids after subtracting the relevant threshold, while partial excursions are split at linearly interpolated crossing points and the out-of-band portion is integrated as one or two triangles. Signal within the interval `[noiseBand_low, noiseBand_high]` is treated as baseline-like noise and contributes zero area.

Before integration, both required columns are coerced to numeric and only finite x-y pairs are retained. If the input is not a data frame with the required columns, or if no finite pairs remain after cleaning, the function returns NA for both outputs to indicate that the curve could not be analyzed. If fewer than two valid points remain, the function returns zero area for both outputs because no segment exists to integrate.

As a practical correction for small transformation offsets near contact, any negative `separation_distance_nm` values are clamped to 0 before integration. This avoids assigning artificial area to small negative-distance tails.

Value

A named numeric vector of length 2:

adhesive_area Total positive area where `y < noiseBand_low`.

repulsive_area Total positive area where `y > noiseBand_high`.

See Also

`area_trapezoid`, `area_triangle`, `crossing_x0`

Examples

```
transformed_df <- data.frame(
  separation_distance_nm = c(
    0.22196440, 0.19896440, 0.17496440, 0.23529773, 0.21129773,
    0.18829773, 0.16429773, 0.14129773, 0.11729773, 0.17763107,
    0.15463107, 0.13063107, 0.19096440, 0.08363107, -0.02270227,
    -0.21336893, -0.31970227, -0.42603560, -0.53336893, -0.47303560,
    -0.49703560, -0.43670227, -0.54403560, -0.65036893, -0.67336893,
    -0.78070227, -0.72036893, -0.49436893, -0.18403560, 0.04196440,
    0.18563107, 0.49496440, 1.13863107, 3.03229773, 5.09163107,
    7.15196440, 8.21129773, 9.27163107, 10.58096440, 11.97463107,
    13.28396440, 14.42763107, 15.48796440, 16.38063107, 17.44096440,
    18.41696440, 19.39396440, 20.28663107, 21.26363107, 22.24063107,
    23.13329773, 24.11029773, 25.08629773, 26.06329773, 27.12263107,
    28.01629773, 28.99329773, 29.96929773, 30.94629773, 31.92229773,
    32.89929773, 33.87529773, 34.85229773, 35.82829773, 36.72196440,
    37.69896440, 38.75829773, 39.73529773, 40.79463107, 41.77163107,
```

```

42.74763107, 43.72463107, 44.70063107, 45.67763107, 46.57129773,
47.54729773, 48.52429773, 49.58363107, 50.47729773, 51.45329773,
52.43029773, 53.49063107, 54.38329773, 55.44363107, 56.33629773,
57.31329773, 58.28929773, 59.34963107, 60.32663107, 61.21929773,
62.19629773, 63.25563107, 64.23263107, 65.37529773, 66.26896440,
67.24496440, 68.13863107, 69.03229773, 70.00829773, 70.90196440
),
force_nN = c(
  2.588500000, 2.488500000, 2.388500000, 2.296833333, 2.196833333,
  2.096833333, 1.996833333, 1.896833333, 1.796833333, 1.705166667,
  1.605166667, 1.505166667, 1.413500000, 1.305166667, 1.196833333,
  1.080166667, 0.971833333, 0.863500000, 0.755166667, 0.663500000,
  0.563500000, 0.471833333, 0.363500000, 0.255166667, 0.155166667,
  0.046833333, -0.044833333, -0.119833333, -0.186500000, -0.261500000,
  -0.344833333, -0.411500000, -0.444833333, -0.353166667, -0.244833333,
  -0.136500000, -0.128166667, -0.119833333, -0.086500000, -0.044833333,
  -0.011500000, 0.005166667, 0.013500000, 0.005166667, 0.013500000,
  0.013500000, 0.013500000, 0.005166667, 0.005166667, 0.005166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, 0.005166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.003166667,
  -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.011500000,
  -0.011500000, -0.003166667, -0.003166667, 0.005166667, 0.005166667,
  0.005166667, 0.005166667, 0.005166667, -0.003166667,
  -0.003166667, -0.003166667, 0.005166667, -0.003166667, -0.003166667,
  -0.003166667, 0.005166667, -0.003166667, 0.005166667, -0.003166667,
  -0.003166667, 0.005166667, 0.005166667, 0.021833333, 0.013500000,
  0.013500000, 0.005166667, -0.003166667, -0.003166667, -0.011500000
)
)
analyze_a_curve_area(transformed_df, noiseBand_low = -0.1, noiseBand_high = 0.1)

```

analyze_a_curve_interaction_distance

Analyze interaction distance from a single AFM curve using noise-band thresholds

Description

Detects a single interaction distance from one transformed AFM curve by searching for the first sustained excursion of force_{nN} outside a user-supplied noise band. The function never scans the final baseline_{span} points, treating that tail region as baseline-only, and instead searches only the portion of the curve before that baseline window. This lets the same function be used for either rupture-type events (negative-force excursions) or repulsive-contact events (positive-force excursions) once appropriate threshold values have already been estimated.

Scan direction is controlled by x_{direction}. With x_{direction} = "left", the search proceeds right-to-left, starting just before the excluded baseline region and moving toward the origin. With x_{direction} = "right", the search proceeds left-to-right, starting at the origin-side of the curve and stopping just before the baseline region. After thresholding the scanned force values, the function uses run-length encoding to identify contiguous stretches of points that satisfy the event condition and accepts only runs of length at least min_{consecutive}.

The reported interaction distance depends on the event type. For y_{direction} = "negative", the function looks for the first accepted run where force_{nN} < noiseBand_{low} and reports the first

point of that run, corresponding to entry into the adhesive or rupture region. For `y_direction = "positive"`, it looks for the first accepted run where `force_nN > noiseBand_high` and reports the last point of that run, corresponding to exit from the repulsive excursion. If no qualifying run is found, or if there is no scannable region before baseline, the function returns NA for distance and echoes back the threshold used.

Usage

```
analyze_a_curve_interaction_distance(
  curve_df,
  baseline_span,
  y_direction = c("negative", "positive"),
  x_direction = c("left", "right"),
  min_consecutive = 1,
  noiseBand_low = NULL,
  noiseBand_high = NULL
)
```

Arguments

<code>curve_df</code>	data.frame with columns: - <code>separation_distance_nm</code> (numeric): x values (distance, nm) - <code>force_nN</code> (numeric): y values (force, nN)
<code>baseline_span</code>	Integer ≥ 1 . Number of last points used as the baseline window. The baseline region will not be scanned.
<code>y_direction</code>	"negative" or "positive". - "negative": find first $y < \text{threshold}$ (rupture-like; threshold is typically negative) - "positive": find the last point of the first run of $y > \text{threshold}$ (repulsion-like; threshold is typically positive)
<code>x_direction</code>	"left" or "right". - "left": scan from right to left (from just before baseline toward the origin) - "right": scan from left to right (from origin up to just before baseline)
<code>min_consecutive</code>	Integer ≥ 1 . Minimum number of consecutive points that must satisfy the threshold criterion for an interaction event to be called.
<code>noiseBand_low</code>	Numeric scalar threshold for negative-direction detection. Required when <code>y_direction = "negative"</code> .
<code>noiseBand_high</code>	Numeric scalar threshold for positive-direction detection. Required when <code>y_direction = "positive"</code> .

Value

A named numeric vector of length 2:

distance Interaction distance in nm for the detected event. Returns `NA_real_` when no qualifying excursion is found or when the input is invalid. Negative distances are clamped to 0.

threshold The numeric threshold actually used for detection: `noiseBand_low` for negative-direction searches or `noiseBand_high` for positive-direction searches. Returns `NA_real_` only when validation fails before threshold selection.

Examples

```

transformed_df <- data.frame(
  separation_distance_nm = c(
    0.22196440, 0.19896440, 0.17496440, 0.23529773, 0.21129773,
    0.18829773, 0.16429773, 0.14129773, 0.11729773, 0.17763107,
    0.15463107, 0.13063107, 0.19096440, 0.08363107, -0.02270227,
    -0.21336893, -0.31970227, -0.42603560, -0.53336893, -0.47303560,
    -0.49703560, -0.43670227, -0.54403560, -0.65036893, -0.67336893,
    -0.78070227, -0.72036893, -0.49436893, -0.18403560, 0.04196440,
    0.18563107, 0.49496440, 1.13863107, 3.03229773, 5.09163107,
    7.15196440, 8.21129773, 9.27163107, 10.58096440, 11.97463107,
    13.28396440, 14.42763107, 15.48796440, 16.38063107, 17.44096440,
    18.41696440, 19.39396440, 20.28663107, 21.26363107, 22.24063107,
    23.13329773, 24.11029773, 25.08629773, 26.06329773, 27.12263107,
    28.01629773, 28.99329773, 29.96929773, 30.94629773, 31.92229773,
    32.89929773, 33.87529773, 34.85229773, 35.82829773, 36.72196440,
    37.69896440, 38.75829773, 39.73529773, 40.79463107, 41.77163107,
    42.74763107, 43.72463107, 44.70063107, 45.67763107, 46.57129773,
    47.54729773, 48.52429773, 49.58363107, 50.47729773, 51.45329773,
    52.43029773, 53.49063107, 54.38329773, 55.44363107, 56.33629773,
    57.31329773, 58.28929773, 59.34963107, 60.32663107, 61.21929773,
    62.19629773, 63.25563107, 64.23263107, 65.37529773, 66.26896440,
    67.24496440, 68.13863107, 69.03229773, 70.00829773, 70.90196440
  ),
  force_nN = c(
    2.588500000, 2.488500000, 2.388500000, 2.296833333, 2.196833333,
    2.096833333, 1.996833333, 1.896833333, 1.796833333, 1.705166667,
    1.605166667, 1.505166667, 1.413500000, 1.305166667, 1.196833333,
    1.080166667, 0.971833333, 0.863500000, 0.755166667, 0.663500000,
    0.563500000, 0.471833333, 0.363500000, 0.255166667, 0.155166667,
    0.046833333, -0.044833333, -0.119833333, -0.186500000, -0.261500000,
    -0.344833333, -0.411500000, -0.444833333, -0.353166667, -0.244833333,
    -0.136500000, -0.128166667, -0.119833333, -0.086500000, -0.044833333,
    -0.011500000, 0.005166667, 0.013500000, 0.005166667, 0.013500000,
    0.013500000, 0.013500000, 0.005166667, 0.005166667, 0.005166667,
    -0.003166667, -0.003166667, -0.003166667, -0.003166667, 0.005166667,
    -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.003166667,
    -0.003166667, -0.003166667, -0.003166667, -0.003166667, -0.011500000,
    -0.011500000, -0.003166667, -0.003166667, 0.005166667, 0.005166667,
    0.005166667, 0.005166667, 0.005166667, -0.003166667,
    -0.003166667, -0.003166667, 0.005166667, -0.003166667, -0.003166667,
    -0.003166667, 0.005166667, -0.003166667, 0.005166667, -0.003166667,
    -0.003166667, 0.005166667, 0.005166667, 0.021833333, 0.013500000,
    0.013500000, 0.005166667, -0.003166667, -0.003166667, -0.011500000
  )
)
rupdis = analyze_a_curve_interaction_distance(transformed_df,
                                              y_direction = "negative",
                                              x_direction = "left",
                                              baseline_span = 50,
                                              noiseBand_low = -0.1,
                                              noiseBand_high = 0.1,
                                              min_consecutive = 3
                                              )
plot(transformed_df$separation_distance_nm, transformed_df$force_nN, type = "l")

```



```

abline(v = rupdis['distance'], col= "red", lwd = 2)

repdis = analyze_a_curve_interaction_distance(transformed_df,
                                              y_direction = "positive",
                                              x_direction = "right",
                                              baseline_span = 50,
                                              noiseBand_low = -0.1,
                                              noiseBand_high = 0.1,
                                              min_consecutive = 3
                                              )
plot(transformed_df$separation_distance_nm, transformed_df$force_nN, type = "l")
abline(v = repdis['distance'], col= "blue", lwd = 2)

```

analyze_a_curve_noise *Determine single-curve noise cutoff from baseline force noise*

Description

Estimates lower and upper force-noise bands from the baseline region of one transformed AFM force-separation curve. The function uses the last `baseline_span` rows of `force_nN` as the baseline window. It then converts that baseline window into a negative and positive threshold pair, `noiseBand_low/noiseBand_high`, that can be used for downstream rupture, repulsion, or area analyses.

Usage

```

analyze_a_curve_noise(
  curve_df,
  baseline_span,
  threshold_method = c("sd", "mad", "quantile", "fixed"),
  multiplier = 3,
  mad_constant = 1.4826,
  quantile_low = 0.25,
  quantile_high = 0.75,
  fixed_low = NULL,
  fixed_high = NULL
)

```

Arguments

<code>curve_df</code>	data.frame with columns: - <code>separation_distance_nm</code> (numeric): x values (distance, nm) - <code>force_nN</code> (numeric): y values (force, nN)
<code>baseline_span</code>	Integer ≥ 1 . Number of last points used as the baseline window.
<code>threshold_method</code>	Character scalar controlling how baseline noise bands are estimated. One of <code>\code{"sd"}</code> , <code>\code{"mad"}</code> , <code>\code{"quantile"}</code> , <code>\code{"fixed"}</code> (default <code>\code{"sd"}</code>): <code>\itemize \item \code{"sd"}: symmetric bands from baseline SD, i.e. $\sqrt{\text{pm}} \cdot \text{sd}(y_{\text{base}}) \cdot \text{multiplier}$. <code>\item \code{"mad"}: symmetric bands from MAD, i.e. $\sqrt{\text{pm}} \cdot \text{mad}(y_{\text{base}}) \cdot \text{mad_constant} \cdot \text{multiplier}$. <code>\item \code{"quantile"}: asymmetric empirical bands using <code>\code{quantile_low}</code> and <code>\code{quantile_high}</code>, each scaled by <code>\code{multiplier}</code>. <code>\item \code{"fixed"}: user-specified <code>\code{fixed_low}</code> and <code>\code{fixed_high}</code>, each scaled by <code>\code{multiplier}</code>.</code></code></code></code>

<code>multiplier</code>	Numeric ≥ 0 scaling multiplier for all methods. Default 3.
<code>mad_constant</code>	Scaling for MAD to be SD-equivalent. Default 1.4826.
<code>quantile_low</code>	Lower quantile probability for "quantile" method. Default 0.25.
<code>quantile_high</code>	Upper quantile probability for "quantile" method. Default 0.75.
<code>fixed_low</code>	Numeric lower band (nN) for "fixed" method. Default NULL.
<code>fixed_high</code>	Numeric upper band (nN) for "fixed" method. Default NULL.

Details

Four thresholding strategies are supported. "sd" builds symmetric bands from the baseline standard deviation, "mad" builds symmetric bands from the median absolute deviation scaled by `mad_constant`, "quantile" uses empirical lower and upper quantiles of the baseline force distribution, and "fixed" uses user-supplied lower and upper force cutoffs directly. In all cases the final bands are multiplied by `multiplier`. For the symmetric methods, if the estimated spread is missing or non-positive, the function substitutes a very small positive value (`.Machine$double.eps`) so that downstream methods still receive non-zero thresholds instead of a degenerate zero-width band.

Invalid inputs such as a non-data-frame `curve_df`, missing required columns, or an invalid `baseline_span` do not stop execution; they return NA noise bands with a warning. By contrast, invalid method-specific arguments such as an out-of-range quantile or missing `fixed_low`/`fixed_high` for "fixed" cause an error, because the requested thresholding rule itself cannot be evaluated.

Value

A named numeric vector of length 2:

noiseBand_low Lower force threshold in nN, typically negative for symmetric methods. Returns `NA_real_` when the curve cannot be analyzed.

noiseBand_high Upper force threshold in nN, typically positive for symmetric methods. Returns `NA_real_` when the curve cannot be analyzed.

Examples

```
transformed_df <- data.frame(
  separation_distance_nm = c(
    0.22196440, 0.19896440, 0.17496440, 0.23529773, 0.21129773,
    0.18829773, 0.16429773, 0.14129773, 0.11729773, 0.17763107,
    0.15463107, 0.13063107, 0.19096440, 0.08363107, -0.02270227,
    -0.21336893, -0.31970227, -0.42603560, -0.53336893, -0.47303560,
    -0.49703560, -0.43670227, -0.54403560, -0.65036893, -0.67336893,
    -0.78070227, -0.72036893, -0.49436893, -0.18403560, 0.04196440,
    0.18563107, 0.49496440, 1.13863107, 3.03229773, 5.09163107,
    7.15196440, 8.21129773, 9.27163107, 10.58096440, 11.97463107,
    13.28396440, 14.42763107, 15.48796440, 16.38063107, 17.44096440,
    18.41696440, 19.39396440, 20.28663107, 21.26363107, 22.24063107,
    23.13329773, 24.11029773, 25.08629773, 26.06329773, 27.12263107,
    28.01629773, 28.99329773, 29.96929773, 30.94629773, 31.92229773,
    32.89929773, 33.87529773, 34.85229773, 35.82829773, 36.72196440,
    37.69896440, 38.75829773, 39.73529773, 40.79463107, 41.77163107,
    42.74763107, 43.72463107, 44.70063107, 45.67763107, 46.57129773,
    47.54729773, 48.52429773, 49.58363107, 50.47729773, 51.45329773,
    52.43029773, 53.49063107, 54.38329773, 55.44363107, 56.33629773,
    57.31329773, 58.28929773, 59.34963107, 60.32663107, 61.21929773,
    62.19629773, 63.25563107, 64.23263107, 65.37529773, 66.26896440,
```


least_length	Either a single integer (minimum number of points in the baseline segment) or "automatic". When "automatic", per-curve span values are read from fdObj@metadata\$baseline_span_approach or fdObj@metadata\$baseline_span_retract depending on useCurve.
useCurve	Character. Either "approach" or "retract" to specify which raw curve to use.
slp_threshold	Numeric. Maximum absolute slope allowed for baseline detection (default = 0.01).
std_threshold	Numeric. Maximum standard error of the slope (default = 0.05).
threads	Integer. Number of parallel threads to use (default = 1).
soft	Logical. If TRUE, use the sensitivity column specified by external_sensitivity_column instead of the default sensitivity_V_nm_approach / sensitivity_V_nm_retract. Default FALSE.
external_sensitivity_column	Character. Name of a column in fdObj@metadata containing sensitivity values. Required when soft = TRUE.

Details

Baseline detection is therefore based on the same criteria implemented in find_baseline(): the last least_length points are tested after scaling the deflection signal by sensitivity, and the window is accepted only if the fitted slope magnitude is below slp_threshold and the slope standard error is below std_threshold. When least_length is a single number, that same trailing-window size is used for all curves; when it is "automatic", per-curve window sizes are taken from the corresponding baseline_span_<dir> metadata column.

Curves with missing required columns, missing sensitivity values, invalid per-curve baseline spans, or failed baseline detection are not dropped; they are recorded with NA baseline values and empty baseline-segment data frames. Successful results are stored in fdObj@metadata\$baseline_V_approach or fdObj@metadata\$baseline_V_retract, and in fdObj@baseline_segment[[useCurve]]. If least_length is fixed, the matching baseline_span_<dir> metadata column is updated to that value for all curves.

Value

An updated fdObj with baseline values in metadata and baseline segments in baseline_segment. If least_length is numeric, the corresponding baseline_span_<dir> column is updated; if least_length = "automatic", existing span columns are preserved.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- analyze_sensitivity(fd_obj, end = 80, intv = 10, useCurve = "retract")
fd_obj <- analyze_baseline(fd_obj, least_length = 100, useCurve = "retract", slp_threshold = 0.02, std_thresho
head(fd_obj@metadata$baseline_V_retract)
```

analyze_curves_adhesive_force

Adhesive Force for All Transformed Curves in an fdObj

Description

Applies `analyze_a_curve_adhesive_force()` to each transformed curve in an `fdObj` and stores results in metadata:

- `adhesive_force_nN<dir>`
- `adhesive_sep_nm<dir>`

Usage

```
analyze_curves_adhesive_force(fdObj, useCurve = "retract", threads = 1)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character. Either "approach" or "retract" (default "retract").
<code>threads</code>	Integer. Number of parallel workers (default 1).

Details

Where `<dir>` is "approach" or "retract".

Behavior:

- If a curve is present and valid but has no negative force, `force = 0` and `sep = NA`.
- If a curve cannot be analyzed at all (missing/invalid transformed data), both are `NA`.

Value

The updated `fdObj`.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_adhesive_force(fd_obj, useCurve = "retract", threads = 1)
print(fd_obj@metadata[, c("adhesive_force_nN_retract", "adhesive_sep_nm_retract")])
```

```
analyze_curves_all_analytical_metrics
```

Run full analytical metrics pipeline on transformed curves in an fdObj

Description

Wrapper around:

- `analyze_curves_noise()`
- `analyze_curves_adhesive_force()`
- `analyze_curves_energy()`
- `analyze_curves_interaction_distance()` (optional negative and/or positive runs)

Usage

```
analyze_curves_all_analytical_metrics(
  fdObj,
  useCurve = c("retract", "approach", "both"),
  threads = 1,
  noise_baseline_span = "automatic",
  noise_threshold_method = c("sd", "mad", "quantile", "fixed"),
  noise_multiplier = 3,
  noise_mad_constant = 1.4826,
  noise_quantile_low = 0.25,
  noise_quantile_high = 0.75,
  noise_fixed_low = NULL,
  noise_fixed_high = NULL,
  analyze_adhesive_force = TRUE,
  analyze_energy = TRUE,
  analyze_rupture_distance = TRUE,
  analyze_rupture_distance_baseline_span = "automatic",
  analyze_rupture_distance_x_direction = c("left", "right"),
  analyze_rupture_distance_min_consecutive = 3,
  analyze_repulsive_distance = TRUE,
  analyze_repulsive_distance_baseline_span = "automatic",
  analyze_repulsive_distance_x_direction = c("right", "left"),
  analyze_repulsive_distance_min_consecutive = 1
)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; one of <code>c("retract", "approach", "both")</code> . Default <code>"both"</code> .
<code>threads</code>	Integer. Number of parallel workers passed to wrapped functions (default 1).
<code>noise_baseline_span</code>	Either a single integer ≥ 1 , or <code>"automatic"</code> . Passed to <code>analyze_curves_noise()</code> .
<code>noise_threshold_method</code>	Character scalar; one of <code>c("sd", "mad", "quantile", "fixed")</code> . Passed to <code>analyze_curves_noise()</code> .

`noise_multiplier`
 Numeric ≥ 0 . Passed to `analyze_curves_noise()`.
`noise_mad_constant`
 Numeric. Passed to `analyze_curves_noise()`.
`noise_quantile_low`
 Numeric in $[0,1]$. Passed to `analyze_curves_noise()`.
`noise_quantile_high`
 Numeric in $[0,1]$. Passed to `analyze_curves_noise()`.
`noise_fixed_low`
 Numeric or NULL. Passed to `analyze_curves_noise()` when method is "fixed".
`noise_fixed_high`
 Numeric or NULL. Passed to `analyze_curves_noise()` when method is "fixed".
`analyze_adhesive_force`
 Logical. If TRUE, runs `analyze_curves_adhesive_force()`. Default TRUE.
`analyze_energy` Logical. If TRUE, runs `analyze_curves_energy()`. Default TRUE.
`analyze_rupture_distance`
 Logical. If TRUE, runs interaction-distance analysis with `y_direction = "negative"`.
 Default TRUE.
`analyze_rupture_distance_baseline_span`
 Either a single integer ≥ 1 , or "automatic". Passed to `negative-direction`
`analyze_curves_interaction_distance()`.
`analyze_rupture_distance_x_direction`
 Character; one of `c("left", "right")`. Default "left". Passed to `negative-direction`
`analyze_curves_interaction_distance()`.
`analyze_rupture_distance_min_consecutive`
 Integer ≥ 1 . Minimum consecutive points below threshold required to call a
 rupture event.
`analyze_repulsive_distance`
 Logical. If TRUE, runs interaction-distance analysis with `y_direction = "positive"`.
 Default TRUE.
`analyze_repulsive_distance_baseline_span`
 Either a single integer ≥ 1 , or "automatic". Passed to `positive-direction`
`analyze_curves_interaction_distance()`.
`analyze_repulsive_distance_x_direction`
 Character; one of `c("right", "left")`. Default "right". Passed to `positive-direction`
`analyze_curves_interaction_distance()`.
`analyze_repulsive_distance_min_consecutive`
 Integer ≥ 1 . Minimum consecutive points above threshold required to call a
 repulsive event.

Details

This function runs the analytical-metrics stage of the workflow as one coordinated wrapper. For each requested curve direction, it first estimates per-curve noise bands, then optionally computes adhesive-force metrics, energy metrics, rupture distances, and repulsive distances using those stored thresholds and any required baseline-span settings. The goal is to let users execute the standard downstream analysis sequence with a single call instead of invoking each metric-specific helper manually.

Processing is performed direction by direction. If `useCurve = "both"`, the function runs the full selected pipeline for approach curves and then repeats it for retract curves, updating `fdObj@metadata`

after each stage. Each wrapped function is still responsible for its own validation and for creating or updating its corresponding metadata columns, so this wrapper mainly coordinates argument routing, optional execution, and call order.

This means the returned object may contain a combination of noise-band, adhesive-force, adhesive and repulsive energy, rupture-distance, and repulsive-distance columns, depending on which optional analyses were enabled. Any curves that cannot be analyzed by a downstream step are retained in the object, with that step's output columns typically populated with NA for the affected rows.

Value

Updated fdObj with analytical metrics written to metadata.

See Also

analyze_curves_noise, analyze_curves_adhesive_force, analyze_curves_energy, analyze_curves_interaction_distance

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj = analyze_curves_all_analytical_metrics(
  fd_obj, # the object
  useCurve = "retract", # run analysis for both approach and retract
  threads = 1,

  # noise band parameters
  noise_baseline_span = "automatic", # use quantile method to define the noise band
  noise_threshold_method = "quantile", # use quantile method to define the noise band
  noise_quantile_low = 0.00, # define the lower end of noise band as minimum y value of the baseline section
  noise_quantile_high = 1, # define the upper end of noise band as maximum y value of the baseline section
  noise_multiplier = 1, # multiply the noise band by 1 to directly use the min-max range of the force in baseline

  # adhesive force
  analyze_adhesive_force = TRUE, # whether to analyze adhesive force

  # adhesive energy and repulsive energy
  analyze_energy = TRUE, # whether to analyze interaction energy (repulsive and adhesive)

  # rupture (adhesive) distance
  analyze_rupture_distance = TRUE, # whether to analyze adhesive/rupture distance
  analyze_rupture_distance_baseline_span = "automatic", # use the baseline section previously defined in curve
  analyze_rupture_distance_x_direction = "left", # scan from right to left to find the first data point at which
  analyze_rupture_distance_min_consecutive = 3, # to reduce false positives from random fluctuations, require a

  # repulsive distance
  analyze_repulsive_distance = TRUE, # whether to analyze repulsive distance
  analyze_repulsive_distance_baseline_span = "automatic", # use the baseline section previously defined in curve
  analyze_repulsive_distance_x_direction = "right", # scan from left to right to find the last data point before
  analyze_repulsive_distance_min_consecutive = 1 # because the curve is expected to start from the repulsive region
)
```

analyze_curves_energy *Adhesive and Repulsive Energy (aJ) for All Transformed Curves in an fdObj*

Description

Applies `analyze_a_curve_area()` to every transformed curve stored in an `fdObj` and writes the resulting out-of-band force-distance areas back into metadata as per-curve energy metrics. For each selected curve, the function reads the corresponding lower and upper noise-band thresholds from metadata, forwards those thresholds together with the transformed force-separation data to `analyze_a_curve_area()`, and stores the two returned areas separately rather than collapsing them into a single net value. Negative-force area below the lower threshold is recorded as adhesive energy, while positive-force area above the upper threshold is recorded as repulsive energy.

Usage

```
analyze_curves_energy(fdObj, useCurve = c("retract", "approach"), threads = 1)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; must be one of <code>c("retract", "approach")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1). Uses future+future.apply when <code>threads > 1</code> .

Details

Results are written to the metadata as:

- `adhesive_energy_aJ_<dir>` — total area where $y < \text{noiseBand_low}$
- `repulsive_energy_aJ_<dir>` — total area where $y > \text{noiseBand_high}$

where `<dir>` is "approach" or "retract".

Energies are reported in **attojoules (aJ)**, since $1 \text{ nNm} = 10^{-18} \text{ J} = 1 \text{ aJ}$.

The wrapper does not recompute thresholds or modify curve geometry itself. It relies on the transformed curves already stored in either `fdObj@approachCurves` or `fdObj@retractCurves` and on existing metadata columns `noiseBand_low_nN_<useCurve>` and `noiseBand_high_nN_<useCurve>`. The underlying single-curve routine preserves point order, and clamps negative separation distances to zero before integration, so those same rules apply here on a curve-by-curve basis.

Behavior:

- If transformed curves are missing entirely for the selected direction, the function creates the output metadata columns and fills them with NA.
- If an individual curve is missing required transformed columns, is empty, or has missing noise-band thresholds, that curve is retained but both output energies are recorded as NA.
- If an individual curve is valid and simply contains no area outside the noise band, the returned adhesive and/or repulsive energy is 0 rather than NA.

This function assumes precomputed noise-band columns already exist in `fdObj@metadata` and uses them directly:

- noiseBand_low_nN_<useCurve>
- noiseBand_high_nN_<useCurve>

Example for useCurve = "retract": noiseBand_low_nN_retract and noiseBand_high_nN_retract.

Value

The updated fdObj with two new metadata columns: adhesive_energy_aJ_<dir> and repulsive_energy_aJ_<dir>.

See Also

analyze_a_curve_area, analyze_curves_adhesive_force

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_noise(fd_obj, useCurve = "retract", threads = 1)
fd_obj = analyze_curves_energy(fd_obj, useCurve = "retract")
```

analyze_curves_interaction_distance

Interaction Distance (nm) for All Transformed Curves in an fdObj

Description

Applies analyze_a_curve_interaction_distance() across all transformed curves stored in an fdObj and writes the detected interaction distances and thresholds back into the metadata. Depending on y_direction, the function measures either rupture-like interaction distances from negative force excursions or repulsive interaction distances from positive force excursions. The selected transformed-curve branch is controlled by useCurve, so the analysis operates on either fdObj@approachCurves or fdObj@retractCurves.

Usage

```
analyze_curves_interaction_distance(
  fdObj,
  useCurve = c("retract", "approach"),
  threads = 1,
  baseline_span,
  y_direction = c("negative", "positive"),
  x_direction = c("left", "right"),
  min_consecutive = 1
)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; must be one of <code>c("retract", "approach")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1).
<code>baseline_span</code>	Either a single integer ≥ 1 , or the string "automatic". When "automatic", per-curve baseline spans are read from <code>fdObj@metadata\$baseline_span_<useCurve></code> .
<code>y_direction</code>	Detection mode: • "negative": detects first point with <code>force_nN < noiseBand_low</code> (curve enters adhesive region; threshold is typically negative). • "positive": detects last point with <code>force_nN > noiseBand_high</code> (curve exits repulsive excursion; threshold is typically positive).
<code>x_direction</code>	Scan direction for each curve: • "left": right-to-left, from just before baseline toward origin. • "right": left-to-right, from origin toward just before baseline.
<code>min_consecutive</code>	Integer ≥ 1 . Minimum number of consecutive points that must satisfy the excursion criterion to call an interaction event.

Details

This wrapper does not estimate thresholds itself. Instead, it expects the relevant per-curve noise-band metadata column to already exist and then uses those stored values as direct inputs to `analyze_a_curve_interaction`. For negative-direction searches it reads `noiseBand_low_nN_<useCurve>`; for positive-direction searches it reads `noiseBand_high_nN_<useCurve>`. Baseline spans can be supplied as one fixed value for all curves or read per curve from `baseline_span_<useCurve>` when `baseline_span = "automatic"`.

Curves are processed independently. For each curve, the function verifies that transformed force-separation data are present, resolves the appropriate baseline span and threshold value, and then forwards the curve together with `x_direction`, `y_direction`, and `min_consecutive` to the single-curve detector. Curves with missing transformed data, missing threshold metadata, or invalid per-curve baseline spans are not dropped but recorded as NA results. After processing, the per-curve outputs are aligned to metadata row order and stored in direction-specific columns.

This function assumes precomputed noise-band columns already exist in `fdObj@metadata` and uses them directly:

- if `y_direction = "negative"`: `noiseBand_low_nN_<useCurve>`
- if `y_direction = "positive"`: `noiseBand_high_nN_<useCurve>`

Example for `useCurve = "retract"`: `noiseBand_low_nN_retract` or `noiseBand_high_nN_retract`.

Value

The updated `fdObj` with two metadata columns added or updated: `<type>_distance_nm_<dir>` and `<type>_threshold_nN_<dir>`, where `<type>` is "repulsive" or "rupture", depending on `y_direction`, and `<dir>` is "approach" or "retract". Curves that cannot be analyzed are retained with NA values in those output columns.

See Also

`analyze_a_curve_interaction_distance`

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_noise(fd_obj, useCurve = "retract", threads = 1)
fd_obj <- analyze_curves_interaction_distance(fd_obj, useCurve = "retract", baseline_span = 'automatic', x_dir = 'x')
```

analyze_curves_noise *Noise-band Analysis (nN) for All Transformed Curves in an fdObj*

Description

Applies `analyze_a_curve_noise()` to each transformed curve in an `fdObj` and stores per-curve lower/upper noise bands in metadata:

- `noiseBand_low_nN<dir>`
- `noiseBand_high_nN<dir>`

where `<dir>` is "approach" or "retract".

Usage

```
analyze_curves_noise(
  fdObj,
  useCurve = c("retract", "approach"),
  threads = 1,
  baseline_span = "automatic",
  threshold_method = c("sd", "mad", "quantile", "fixed"),
  multiplier = 3,
  mad_constant = 1.4826,
  quantile_low = 0.25,
  quantile_high = 0.75,
  fixed_low = NULL,
  fixed_high = NULL
)
```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>useCurve</code>	Character; must be one of <code>c("retract", "approach")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1).
<code>baseline_span</code>	Either a single integer ≥ 1 , or the string "automatic". When "automatic", per-curve baseline spans are read from <code>fdObj@metadata\$baseline_span<useCurve></code> .
<code>threshold_method</code>	Character scalar controlling how baseline noise bands are estimated. One of <code>c("sd", "mad", "quantile", "fixed")</code> (default "sd").
<code>multiplier</code>	Numeric ≥ 0 scaling multiplier for all methods. Default 3.
<code>mad_constant</code>	Scaling for MAD to be SD-equivalent. Default 1.4826.

quantile_low	Lower quantile probability for "quantile" method. Default 0.25.
quantile_high	Upper quantile probability for "quantile" method. Default 0.75.
fixed_low	Numeric lower band (nN) for "fixed" method. Default NULL.
fixed_high	Numeric upper band (nN) for "fixed" method. Default NULL.

Value

The updated fdObj with two new metadata columns: noiseBand_low_nN_<dir> and noiseBand_high_nN_<dir>.

See Also

analyze_a_curve_noise

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_noise(fd_obj, useCurve = "retract", threads = 1)
```

analyze_sensitivity *Add Sensitivity Estimates to an fdObj*

Description

Applies sensitivity estimation to each rawCurve in an fdObj and stores the sensitivity values in the metadata and the segment used in the senscal_segment slot.

Usage

```
analyze_sensitivity(
  fdObj,
  end = 200,
  intv = 4,
  R_squared_min = 0.99,
  minimum_length = 4,
  useCurve = "approach",
  threads = 1
)
```

Arguments

fdObj	An object of class fdObj.
end	Integer. The maximum index in raw curves to consider (e.g., 200) in calc_sensitivity().
intv	Integer. Number of data points added per iteration when building the linear sensitivity segment in calc_sensitivity().
R_squared_min	Numeric. Minimum R^2 threshold for accepting a segment as sufficiently linear during sensitivity calculation in calc_sensitivity(). Default is 0.99.
minimum_length	Integer. Minimum number of accumulated points required for a valid sensitivity result. If the segment is shorter, sensitivity is reported as NA. Passed to calc_sensitivity(). Default is 4.

useCurve Character. Either "approach" or "retract" to determine which curve to use.
 threads Number of parallel threads to use (default = 1).

Value

An updated fdObj with sensitivity values in metadata and segments in senscal_segment.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- analyze_sensitivity(fd_obj, end = 80, intv = 10, useCurve = "retract")
head(fd_obj@metadata$sensitivity_V_nm_retract)
```

area_trapezoid	<i>Calculate the trapezoid area between two points</i>
----------------	--

Description

Computes the absolute area of a trapezoid formed by two points (x1, y1) and (x2, y2) along the x-axis. This is used when both points are on the same side of the x-axis (either both positive or both negative).

Usage

```
area_trapezoid(x1, y1, x2, y2)
```

Arguments

x1 Numeric. The x-coordinate of the first point.
 y1 Numeric. The y-coordinate of the first point.
 x2 Numeric. The x-coordinate of the second point.
 y2 Numeric. The y-coordinate of the second point.

Value

A single numeric value representing the positive trapezoid area between the two points.

See Also

[area_triangle()], [crossing_x0()], [calculate_area()]

Examples

```
curvana:::area_trapezoid(0, 2, 1, 4)    # 0.5 * (1 - 0) * (2 + 4) = 3
curvana:::area_trapezoid(0, -2, 1, -4) # same absolute area = 3
```

area_triangle	<i>Calculate the triangle area between a curve point and the x-axis</i>
---------------	---

Description

Computes the absolute area of a triangle formed by a point (x1, y1) on the curve and its intersection with the x-axis at (x0, 0). This function assumes one vertex lies on the x-axis (y = 0).

Usage

```
area_triangle(x1, y1, x0)
```

Arguments

x1	Numeric. The x-coordinate of the curve point.
y1	Numeric. The y-coordinate of the curve point.
x0	Numeric. The x-coordinate of the x-axis intersection point (where y = 0).

Value

A single numeric value representing the positive triangle area.

See Also

[area_trapezoid()], [crossing_x0()], [calculate_area()]

Examples

```
# Triangle between (0, 4) and (1, 0)
curvana:::area_triangle(0, 4, 1) # 0.5 * |1 - 0| * |4| = 2
```

calc_sensitivity	<i>Calculate Sensitivity from a Segment of AFM Data</i>
------------------	---

Description

This function identifies a linear segment from AFM deflection vs. piezo extension data and calculates the sensitivity (slope) using linear regression. It iteratively adds chunks of data to build a segment with high linearity ($R^2 > R_squared_min$) and optionally removes outliers to maintain this criterion.

Usage

```
calc_sensitivity(x, y, R_squared_min = 0.99, end, intv = 4, minimum_length = 4)
```

Arguments

<code>x</code>	Numeric vector. The piezo extension values (e.g., distance in nm).
<code>y</code>	Numeric vector. The deflection values (e.g., voltage).
<code>R_squared_min</code>	Numeric. Minimum R^2 threshold for accepting a segment as sufficiently linear during sensitivity calculation. Default is 0.99.
<code>end</code>	Integer. The maximum index to consider in the 'x' and 'y' vectors (i.e., up to which point to search).
<code>intv</code>	Integer. Number of data points added per iteration when building the linear sensitivity segment.
<code>minimum_length</code>	Integer. Minimum number of accumulated points required for a valid sensitivity result. If the segment is shorter, sensitivity is reported as NA. Default is 4.

Value

A list of three elements:

senscal_segment a dataframe of a subset of points in the original raw curve which are used for sensitivity calibration

sensitivity The calculated sensitivity (slope) as a numeric value, or NULL if not computable.

Examples

```
x <- c(
  0.000, 0.977, 1.953, 2.930, 3.906, 4.883, 5.859, 6.836, 7.812, 8.789,
  9.766, 10.742, 11.719, 12.695, 13.672, 14.648, 15.625, 16.602, 17.578, 18.555,
  19.531, 20.508, 21.484, 22.461, 23.438, 24.414, 25.391, 26.367, 27.344, 28.320,
  29.297, 30.273, 31.250, 32.227, 33.203, 34.180, 35.156, 36.133, 37.109, 38.086,
  39.062, 40.039, 41.016, 41.992, 42.969, 43.945, 44.922, 45.898, 46.875, 47.852,
  48.828, 49.805, 50.781, 51.758, 52.734, 53.711, 54.688, 55.664, 56.641, 57.617,
  58.594, 59.570, 60.547, 61.523, 62.500, 63.477, 64.453, 65.430, 66.406, 67.383,
  68.359, 69.336, 70.312, 71.289, 72.266, 73.242, 74.219, 75.195, 76.172, 77.148,
  78.125, 79.102, 80.078, 81.055, 82.031, 83.008, 83.984, 84.961, 85.938, 86.914,
  87.891, 88.867, 89.844, 90.820, 91.797, 92.773, 93.750, 94.727, 95.703, 96.680
)

y <- c(
  0.255, 0.243, 0.231, 0.220, 0.208, 0.196, 0.184, 0.172, 0.160, 0.149,
  0.137, 0.125, 0.114, 0.101, 0.088, 0.074, 0.061, 0.048, 0.035, 0.024,
  0.012, 0.001, -0.012, -0.025, -0.037, -0.050, -0.061, -0.070, -0.078, -0.087,
  -0.097, -0.105, -0.109, -0.098, -0.085, -0.072, -0.071, -0.070, -0.066, -0.061,
  -0.057, -0.055, -0.054, -0.055, -0.054, -0.054, -0.054, -0.055, -0.055, -0.055,
  -0.056, -0.056, -0.056, -0.056, -0.055, -0.056, -0.056, -0.056, -0.056, -0.056,
  -0.056, -0.056, -0.056, -0.056, -0.057, -0.057, -0.056, -0.056, -0.055, -0.055,
  -0.055, -0.055, -0.055, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
  -0.056, -0.055, -0.056, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
  -0.056, -0.055, -0.055, -0.053, -0.054, -0.054, -0.055, -0.056, -0.056, -0.057
)

calc_sensitivity(x = x, y = y, end = 80, intv = 10)
```

combineFdObj	<i>Combine two fdObj objects</i>
--------------	----------------------------------

Description

Combine two fdObj objects

Usage

```
combineFdObj(x, y)
```

Arguments

x	A 'fdObj' object
y	A 'fdObj' object. This object cannot have any overlapping sample names with the first fdObj.

Value

A new combined 'fdObj' object. metadata matrix will be concatenated using bind_rows (dplyr). All slots will be concatenated.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"
md$surface <- surface
fd_obj@metadata <- md
fd_obj2 = extract(fd_obj,by_col = list(surface = 'Group_B'))
fd_obj3 = extract(fd_obj,by_sample = 'F_1.spm-F4E1_ForceCurveIndex_0.spm')
fd_obj4 = combineFdObj(fd_obj2, fd_obj3)
```

createFdObjFromFolder	<i>Constructor for fdObj from a folder of AFM force-distance curve files</i>
-----------------------	--

Description

This function imports raw AFM force-distance curve files where cantilever deflection (in Voltage) and piezo displacement are stored as separate columns for approach and retract segments, such as files exported from Veeco and Bruker Nanoscope softwares. Each curve should have the contact region at the beginning of the curve and the non-interaction region at the end of the curve, so that the approach and retract segments can be correctly identified and processed. However, different instruments generate curves with different column names, orientations, and conventions, so the user must specify which columns correspond to approach and retract distances and deflections, and whether any of these columns need to be reversed to ensure the contact and non-interaction regions are correctly positioned at the beginning and end of the curve, respectively.

Usage

```
createFdObjFromFolder(
  folder,
  suffix = ".txt",
  pattern = "",
  Displacement_Approach = "Calc_Ramp_Ex_nm",
  Displacement_Retract = "Calc_Ramp_Rt_nm",
  Deflection_Approach = "Defl_V_Ex",
  Deflection_Retract = "Defl_V_Rt",
  metadata = NULL,
  threads = 1,
  reverse_Displacement_Approach = FALSE,
  reverse_Displacement_Retract = TRUE,
  reverse_Deflection_Approach = TRUE,
  reverse_Deflection_Retract = FALSE
)
```

Arguments

folder	Path to a folder containing the raw AFM curve files
suffix	File extension to look for (e.g., ".txt")
pattern	Optional string pattern to filter files (e.g., "experiment"); default is "" (no filtering)
Displacement_Approach	Column name for approach distance (default: "Calc_Ramp_Ex_nm")
Displacement_Retract	Column name for retract distance (default: "Calc_Ramp_Rt_nm")
Deflection_Approach	Column name for approach deflection (default: "Defl_V_Ex")
Deflection_Retract	Column name for retract deflection (default: "Defl_V_Rt")
metadata	Optional data.frame. If provided, file names (excluding the suffix) matched to rownames(metadata) will be read in. Returns error if any files not found.
threads	Number of parallel threads to use for file reading. Default = 1 (sequential).
reverse_Displacement_Approach	Logical; reverse approach distance column if TRUE. Default = FALSE.
reverse_Displacement_Retract	Logical; reverse retract distance column if TRUE. Default = TRUE.
reverse_Deflection_Approach	Logical; reverse approach deflection column if TRUE. Default = TRUE.
reverse_Deflection_Retract	Logical; reverse retract deflection column if TRUE. Default = FALSE.

Details

The function automatically counts the number of non-NA data points in each segment and adds `Number_of_datapoints_approach` and `Number_of_datapoints_retract` columns to the metadata. These counts can be useful for quality control and for identifying segments with incomplete data.

Value

An object of class fdObj

```
createFdObjFromJPKFolder
```

Constructor for fdObj from a folder of JPK format AFM force-distance curve files

Description

This function reads JPK ASCII format force-distance curve files exported from JPK AFM software. The function automatically detects and separates approach and retract segments, and extracts sensitivity and spring constant. Users are encouraged to export deflection data directly in Voltage. However, if force data is provided (indicated by "N" as the column unit), it will be converted back to Voltage using the associated sensitivity and spring constant values.

Usage

```
createFdObjFromJPKFolder(
  folder,
  suffix = ".txt",
  pattern = "",
  metadata = NULL,
  threads = 1,
  height_col = "height",
  deflection_col = "vDeflection",
  reverse_Displacement_Approach = TRUE,
  reverse_Displacement_Retract = FALSE,
  reverse_Deflection_Approach = TRUE,
  reverse_Deflection_Retract = FALSE
)
```

Arguments

folder	Path to a folder containing JPK format AFM curve files (.txt)
suffix	File extension to look for (default: ".txt")
pattern	Optional string pattern to filter files (e.g., "experiment"); default is "" (no filtering)
metadata	Optional data.frame. If provided, file names (excluding the suffix) matched to rownames(metadata) will be read in. Returns error if any files not found.
threads	Number of parallel threads to use for file reading. Default = 1 (sequential).
height_col	Column name for height/distance data in JPK files. Default is "height".
deflection_col	Column name for deflection data in JPK files. Default is "vDeflection".
reverse_Displacement_Approach	Logical; reverse approach distance column if TRUE. Default = TRUE
reverse_Displacement_Retract	Logical; reverse retract distance column if TRUE. Default = FALSE.
reverse_Deflection_Approach	Logical; reverse approach deflection column if TRUE. Default = TRUE
reverse_Deflection_Retract	Logical; reverse retract deflection column if TRUE. Default = FALSE.

Details

The function reads JPK text export files and automatically:

- Detects approach (extend) and retract segments
- Extracts sensitivity and spring constant separately for each segment
- Determines if deflection data is in Voltage (V) or Force (N)
- If Force (N), converts to Voltage using: $V = F / (\text{sensitivity} * \text{springConstant})$
- Converts distance from meters to nanometers
- Optionally reverses approach/retract distance and deflection columns (Approach: all TRUE by default; Retract: all FALSE by default) to ensure contact region is at the beginning and non-interaction region is at the end of the curve
- Adds segment-specific columns to metadata: approach_sensitivity_imported, approach_springConstant_imported, retract_sensitivity_imported, retract_springConstant_imported, Number_of_datapoints_approach, Number_of_datapoints_retract

The returned fdObj contains:

- rawCurves: Raw data with approach and retract columns (distance in nm, deflection in V). If segments have different lengths, shorter ones are padded with NA. Analysis functions use Number_of_datapoints_* columns to exclude padded NA values.
- approachCurves: Empty initially (to be filled by transform_curves)
- retractCurves: Empty initially (to be filled by transform_curves)
- metadata: Sample information including segment-specific imported sensitivity and spring constant, plus data point counts

Value

An object of class fdObj

Examples

```
## Not run:
# Read all JPK files from a folder
fd_obj <- createFdObjFromJPKFolder("path/to/jpk/files")

# Read files matching a pattern with metadata
metadata <- data.frame(
  sample_type = c("control", "treated"),
  row.names = c("file1", "file2")
)
fd_obj <- createFdObjFromJPKFolder("path/to/jpk/files", metadata = metadata)

# Check imported parameters (segment-specific)
head(fd_obj@metadata[, c("approach_sensitivity_imported",
                        "approach_springConstant_imported",
                        "retract_sensitivity_imported",
                        "retract_springConstant_imported",
                        "Number_of_datapoints_approach",
                        "Number_of_datapoints_retract")])

## End(Not run)
```

crossing_x0	<i>Interpolate the x-intercept (x0) where y = 0</i>
-------------	---

Description

Performs linear interpolation between two points (x1, y1) and (x2, y2) to find the x-coordinate (x0) where the line crosses the x-axis (y = 0). Only valid when y1 and y2 have opposite signs.

Usage

```
crossing_x0(x1, y1, x2, y2)
```

Arguments

x1	Numeric. The x-coordinate of the first point.
y1	Numeric. The y-coordinate of the first point.
x2	Numeric. The x-coordinate of the second point.
y2	Numeric. The y-coordinate of the second point.

Value

A single numeric value for the interpolated x0 coordinate. Returns NA if no valid sign change exists between y1 and y2.

See Also

[area_trapezoid()], [area_triangle()], [calculate_area()]

Examples

```
curvana:::crossing_x0(0, 4, 2, -2) # 1.333 - line crosses y=0 between x=0 and x=2
curvana:::crossing_x0(0, 4, 2, 6)  # NA - no sign change
```

denoise_a_curve	<i>Denoise Deflection Columns in a Raw Curve Data Frame</i>
-----------------	---

Description

Applies Savitzky-Golay smoothing to deflection columns in a raw curve data frame. The original values are first copied to new columns with suffix `_original`, then the target columns are overwritten with smoothed values. If segment length information is provided via `n_approach` and `n_retract`, only the valid (non-NA) portion of each segment is denoised, and the result is padded back to the original length with NA.

Usage

```
denoise_a_curve(
  raw_curve,
  p = 1,
  n = 3,
  m = 0,
  ts = 1,
  useCurve = c("retract", "approach", "both"),
  n_approach = NULL,
  n_retract = NULL
)
```

Arguments

<code>raw_curve</code>	A raw curve data frame.
<code>p</code>	Filter polynomial order.
<code>n</code>	Filter length (must be odd).
<code>m</code>	Order of the derivative to compute.
<code>ts</code>	Sampling interval.
<code>useCurve</code>	Character; one of <code>c("retract", "approach", "both")</code> .
<code>n_approach</code>	Optional. Number of valid data points in approach segment. If provided, only the first <code>n_approach</code> rows of approach columns are denoised.
<code>n_retract</code>	Optional. Number of valid data points in retract segment. If provided, only the first <code>n_retract</code> rows of retract columns are denoised.

Value

A data frame with original columns showing for the smoothed deflection values and additional `*_original` as backup column(s) for the raw deflection values (before denoising).

Examples

```
x <- c(
  0.000, 0.977, 1.953, 2.930, 3.906, 4.883, 5.859, 6.836, 7.812, 8.789,
  9.766, 10.742, 11.719, 12.695, 13.672, 14.648, 15.625, 16.602, 17.578, 18.555,
  19.531, 20.508, 21.484, 22.461, 23.438, 24.414, 25.391, 26.367, 27.344, 28.320,
  29.297, 30.273, 31.250, 32.227, 33.203, 34.180, 35.156, 36.133, 37.109, 38.086,
  39.062, 40.039, 41.016, 41.992, 42.969, 43.945, 44.922, 45.898, 46.875, 47.852,
  48.828, 49.805, 50.781, 51.758, 52.734, 53.711, 54.688, 55.664, 56.641, 57.617,
  58.594, 59.570, 60.547, 61.523, 62.500, 63.477, 64.453, 65.430, 66.406, 67.383,
  68.359, 69.336, 70.312, 71.289, 72.266, 73.242, 74.219, 75.195, 76.172, 77.148,
  78.125, 79.102, 80.078, 81.055, 82.031, 83.008, 83.984, 84.961, 85.938, 86.914,
  87.891, 88.867, 89.844, 90.820, 91.797, 92.773, 93.750, 94.727, 95.703, 96.680
)

y <- c(
  0.255, 0.243, 0.231, 0.220, 0.208, 0.196, 0.184, 0.172, 0.160, 0.149,
  0.137, 0.125, 0.114, 0.101, 0.088, 0.074, 0.061, 0.048, 0.035, 0.024,
  0.012, 0.001, -0.012, -0.025, -0.037, -0.050, -0.061, -0.070, -0.078, -0.087,
  -0.097, -0.105, -0.109, -0.098, -0.085, -0.072, -0.071, -0.070, -0.066, -0.061,
  -0.057, -0.055, -0.054, -0.055, -0.054, -0.054, -0.054, -0.055, -0.055, -0.055,
  -0.056, -0.056, -0.056, -0.056, -0.055, -0.056, -0.056, -0.056, -0.056, -0.056,
```

```

-0.056, -0.056, -0.056, -0.056, -0.057, -0.057, -0.056, -0.056, -0.055, -0.055,
-0.055, -0.055, -0.055, -0.055, -0.056, -0.056, -0.056, -0.055, -0.056, -0.056,
-0.056, -0.055, -0.056, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
-0.056, -0.055, -0.055, -0.053, -0.054, -0.054, -0.055, -0.056, -0.056, -0.057
)
denoised_df <- denoise_a_curve(data.frame(Calc_Ramp_Ex_nm = x, Defl_V_Ex = y), p = 1, n = 5, m = 0, ts = 1, useCu
head(denoised_df)

```

denoise_curves

*Denoise All Raw Curves in an fdObj***Description**

Applies `denoise_a_curve()` to every data frame in `fdObj@rawCurves` and writes the denoised results back to the `rawCurves` slot.

Usage

```

denoise_curves(
  fdObj,
  p = 1,
  n = 3,
  m = 0,
  ts = 1,
  useCurve = c("retract", "approach", "both"),
  threads = 1
)

```

Arguments

<code>fdObj</code>	An object of class <code>fdObj</code> .
<code>p</code>	Filter polynomial order.
<code>n</code>	Filter length (must be odd).
<code>m</code>	Order of the derivative to compute.
<code>ts</code>	Sampling interval.
<code>useCurve</code>	Character; one of <code>c("retract", "approach", "both")</code> .
<code>threads</code>	Integer. Number of parallel workers (default 1).

Value

Updated `fdObj` with denoised curves in `rawCurves`.

Examples

```

folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- denoise_curves(fd_obj, p = 1, n = 5, m = 0, ts = 1, useCurve = "approach", threads = 1)
head(fd_obj@rawCurves[[1]])

```

extract	<i>Extract subset of fdObj by sample or metadata column</i>
---------	---

Description

Extract subset of fdObj by sample or metadata column

Usage

```
extract(fdObj, by_sample = NULL, by_col = NULL)
```

Arguments

fdObj	An object of class fdObj
by_sample	A character vector of sample names to keep
by_col	A named list like list(colname = value) to filter metadata (e.g., list(type = "control"))

Value

A subsetted fdObj object

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"
md$surface <- surface
fd_obj@metadata <- md
fd_obj2 = extract(fd_obj, by_col = list(surface = 'Group_A'))
fd_obj3 = extract(fd_obj, by_sample = 'F_1.spm-F4E1_ForceCurveIndex_0.spm')
```

fdObj-class	<i>An S4 class to hold AFM force-distance raw and processed curves with metadata</i>
-------------	--

Description

‘fdObj’ is the main container class used throughout the package to keep a set of AFM curves and their derived results together in one validated object. Rather than passing raw curves, transformed curves, calibration segments, and metadata as separate objects, ‘fdObj’ stores them in aligned slots so that every sample keeps the same identity across import, transformation, visualization, and analysis steps.

In a typical workflow, an ‘fdObj’ is first created from a folder of raw curve files. At that stage, the object mainly contains imported raw curves and a metadata table. Downstream functions then

populate additional slots with transformed approach and retract curves, curve regions used for sensitivity and baseline estimation, and analytical results written back into the metadata. This makes 'fdObj' the central analysis object for the package and allows most functions to take an 'fdObj', update its contents, and return the modified object.

The class also enforces consistency between its parts. Curve lists must stay named consistently, transformed curves must have the expected structure, and metadata rows must match the registered curve names. This validity checking helps prevent analyses from being run on mismatched or incomplete datasets.

Slots

rawCurves A named list of data.frames containing raw AFM data. Each data.frame must include at least one complete distance-deflection pair: either approach columns Calc_Ramp_Ex_nm and Defl_V_Ex, or retract columns Calc_Ramp_Rt_nm and Defl_V_Rt; both pairs are also allowed. Additional columns are allowed.

approachCurves A named list of 2-column data.frames (distance vs force from approach segment)

retractCurves A named list of 2-column data.frames (distance vs force from retract segment)

metadata A data.frame with row names matching names(rawCurves). Stores metadata and calculated features.

senscal_segment A list with two slots: approach and retract, each is a named list of data.frames, storing the part of the raw curve used for sensitivity calculation.

baseline_segment A list with two slots: approach and retract, each is a named list of data.frames, storing the part of the raw curve used for baseline calculation

find_baseline

Identify Baseline Segment in AFM Raw Force-Distance Curve

Description

This function tests the trailing part of an AFM (Atomic Force Microscopy) raw force-distance curve for a baseline segment. It takes the last `least_length` points after scaling `y` by `sensitivity`, fits a linear model of scaled deflection against `x`, and accepts that trailing window as baseline only when two criteria are met: the absolute fitted slope is smaller than `slp_threshold` and the standard error of that slope is smaller than `std_threshold`. If both criteria are satisfied, the function returns the mean baseline value in the original `y` units and the accepted segment. If either criterion fails, or if the inputs are too short or mismatched, no baseline is reported and `NULL` values are returned.

Usage

```
find_baseline(
  x,
  y,
  least_length,
  sensitivity,
  slp_threshold = 0.01,
  std_threshold = 0.05
)
```

Arguments

x	Numeric vector. Distance values (e.g., piezo positions or z-sensor values).
y	Numeric vector. Deflection values (e.g., in volts).
least_length	Integer. Minimum number of points in the baseline segment. The function will take the last least_length points of the curve for testing.
sensitivity	Numeric. Scaling factor for the deflection signal (e.g., probe sensitivity)).
slp_threshold	Numeric. Maximum absolute slope for the segment to be considered flat (default: 0.01).
std_threshold	Numeric. Maximum standard error of the slope (default: 0.05).

Details

The function assumes the baseline occurs toward the tail end of the curve. It does not search the full curve for candidate windows; it evaluates only the final trailing window defined by least_length.

Value

A list with two elements:

baseline Numeric. The estimated baseline deflection value (in original units), or NULL if no valid segment is found.

segment Data frame with x and y columns representing the detected baseline segment, or NULL if not found.

Examples

```
# Simulated flat tail data
x <- c(
  0.000, 0.977, 1.953, 2.930, 3.906, 4.883, 5.859, 6.836, 7.812, 8.789,
  9.766, 10.742, 11.719, 12.695, 13.672, 14.648, 15.625, 16.602, 17.578, 18.555,
  19.531, 20.508, 21.484, 22.461, 23.438, 24.414, 25.391, 26.367, 27.344, 28.320,
  29.297, 30.273, 31.250, 32.227, 33.203, 34.180, 35.156, 36.133, 37.109, 38.086,
  39.062, 40.039, 41.016, 41.992, 42.969, 43.945, 44.922, 45.898, 46.875, 47.852,
  48.828, 49.805, 50.781, 51.758, 52.734, 53.711, 54.688, 55.664, 56.641, 57.617,
  58.594, 59.570, 60.547, 61.523, 62.500, 63.477, 64.453, 65.430, 66.406, 67.383,
  68.359, 69.336, 70.312, 71.289, 72.266, 73.242, 74.219, 75.195, 76.172, 77.148,
  78.125, 79.102, 80.078, 81.055, 82.031, 83.008, 83.984, 84.961, 85.938, 86.914,
  87.891, 88.867, 89.844, 90.820, 91.797, 92.773, 93.750, 94.727, 95.703, 96.680
)

y <- c(
  0.255, 0.243, 0.231, 0.220, 0.208, 0.196, 0.184, 0.172, 0.160, 0.149,
  0.137, 0.125, 0.114, 0.101, 0.088, 0.074, 0.061, 0.048, 0.035, 0.024,
  0.012, 0.001, -0.012, -0.025, -0.037, -0.050, -0.061, -0.070, -0.078, -0.087,
  -0.097, -0.105, -0.109, -0.098, -0.085, -0.072, -0.071, -0.070, -0.066, -0.061,
  -0.057, -0.055, -0.054, -0.055, -0.054, -0.054, -0.054, -0.055, -0.055, -0.055,
  -0.056, -0.056, -0.056, -0.056, -0.055, -0.056, -0.056, -0.056, -0.056, -0.056,
  -0.056, -0.056, -0.056, -0.056, -0.057, -0.057, -0.056, -0.056, -0.055, -0.055,
  -0.055, -0.055, -0.055, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
  -0.056, -0.055, -0.055, -0.053, -0.054, -0.054, -0.055, -0.056, -0.056, -0.057
)

result <- find_baseline(x,y, least_length = 50,sensitivity = 0.012,slp_threshold = 0.02, std_threshold = 0.02)
```

```
print(result$baseline)
plot(x, y, type = "l"); lines(result$segment, col = "red", lwd = 2)
```

```
plot_all_curve_metrics
```

Plot and save curve metrics for all curves in an fdObj

Description

Takes an fdObj plus plotting and file-output settings as input, iteratively calls plot_a_curve_metrics() for every curve in the selected transformed-curve slot, and saves each generated plot to disk in the requested format. This function is a batch-export wrapper for per-curve metric plots rather than a plotting function for a single curve. Its output is a data frame summarizing which curves were saved successfully, where each file was written, and any error messages for failed plots.

Usage

```
plot_all_curve_metrics(
  fdobj,
  useCurve = c("retract", "approach"),
  threads = 1,
  plot_raw = FALSE,
  annotate_noiseBand = TRUE,
  annotate_repulsive_energy = TRUE,
  annotate_adhesive_energy = TRUE,
  annotate_adhesive_force = TRUE,
  annotate_rupture_distance = TRUE,
  annotate_repulsive_distance = TRUE,
  xlim = NULL,
  ylim = NULL,
  title = NULL,
  base_size = 14,
  line_color = "grey35",
  point_color = "grey35",
  line_size = 0.7,
  point_size = 1.8,
  line_alpha = 0.9,
  point_alpha = 0.8,
  annotation_text_size = 3.3,
  interaction_label_color_rupture = "purple4",
  interaction_label_color_repulsive = "darkorange3",
  interaction_label_fill = "white",
  destination_folder = NULL,
  width = 12,
  height = 6,
  format = c("png", "pdf", "jpeg", "tiff", "bmp", "svg"),
  dpi = 300,
  units = c("in", "cm", "mm"),
  ...
)
```

Arguments

fdobj	An object of class fdObj.
useCurve	Character. Either "retract" or "approach".
threads	Integer. Number of parallel workers (default 1). Uses future+future.apply when threads > 1.
plot_raw	Logical. Passed to plot_a_curve_metrics().
annotate_noiseBand	Logical. Passed to plot_a_curve_metrics().
annotate_repulsive_energy	Logical. Passed to plot_a_curve_metrics().
annotate_adhesive_energy	Logical. Passed to plot_a_curve_metrics().
annotate_adhesive_force	Logical. Passed to plot_a_curve_metrics().
annotate_rupture_distance	Logical. Passed to plot_a_curve_metrics().
annotate_repulsive_distance	Logical. Passed to plot_a_curve_metrics().
xlim	Optional numeric vector of length 2 for x-axis limits.
ylim	Optional numeric vector of length 2 for y-axis limits.
title	Character. Plot title passed to plot_a_curve_metrics().
base_size	Numeric. Base theme text size.
line_color	Character. Line color for curves.
point_color	Character. Point color for curves.
line_size	Numeric. Line width for curves.
point_size	Numeric. Point size for curves.
line_alpha	Numeric. Line alpha for curves.
point_alpha	Numeric. Point alpha for curves.
annotation_text_size	Numeric. Text size for annotation labels.
interaction_label_color_rupture	Character. Label color for rupture annotations.
interaction_label_color_repulsive	Character. Label color for repulsive annotations.
interaction_label_fill	Character. Fill color for interaction labels.
destination_folder	Character. Output folder path. If NULL, a timestamped folder is created in the current working directory.
width	Numeric. Plot width passed to ggplot2::ggsave().
height	Numeric. Plot height passed to ggplot2::ggsave().
format	Character. Output graphics format. One of "png", "pdf", "jpeg", "tiff", "bmp", "svg".
dpi	Numeric. Resolution passed to ggplot2::ggsave().
units	Character. Units for width/height. One of "in", "cm", "mm".
...	Additional ggplot2 components passed to plot_a_curve_metrics().

Value

A data.frame with one row per curve and columns: curve_name, file_path, status, and error_message. The output folder is attached as attr(result, "destination_folder").

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_all_analytical_metrics(fd_obj, useCurve = "retract")
plot_all_curve_metrics(fdobj = fd_obj, useCurve = "retract", plot_raw = T, destination_folder = '../test_plot_m
```

plot_a_curve_metrics *Plot one transformed curve with optional metadata annotations*

Description

Takes an fdObj, the name of one curve, and the selected transformed branch (approach or retract) as input, then builds a detailed visualization of that single transformed force-distance curve. The function can annotate noise-band bounds, adhesive force, adhesive and repulsive energies, rupture distance, repulsive distance, and region labels derived from stored metadata. When plot_raw = TRUE, it also plots the matching raw curve and returns a two-panel display. The output is therefore either one annotated ggplot2 object or a combined cowplot panel containing both raw and transformed views.

Usage

```
plot_a_curve_metrics(
  fdobj,
  curve_name,
  useCurve = c("retract", "approach"),
  plot_raw = FALSE,
  annotate_noiseBand = TRUE,
  annotate_repulsive_energy = TRUE,
  annotate_adhesive_energy = TRUE,
  annotate_adhesive_force = TRUE,
  annotate_rupture_distance = TRUE,
  annotate_repulsive_distance = TRUE,
  xlim = NULL,
  ylim = NULL,
  title = NULL,
  base_size = 14,
  line_color = "grey35",
  point_color = "grey35",
  line_size = 0.7,
  point_size = 1.8,
  line_alpha = 0.9,
  point_alpha = 0.8,
  annotation_text_size = 3.3,
  interaction_label_color_rupture = "purple4",
  interaction_label_color_repulsive = "darkorange3",
```

```

    interaction_label_fill = "white",
    ...
)

```

Arguments

<code>fdobj</code>	An object of class <code>fdObj</code> .
<code>curve_name</code>	Character. Name of one curve to plot. Must match <code>rownames(fdobj@metadata)</code> .
<code>useCurve</code>	Character. Either "retract" or "approach".
<code>plot_raw</code>	Logical. If TRUE, also plot the corresponding raw curve and return a two-column panel using <code>cowplot::plot_grid()</code> .
<code>annotate_noiseBand</code>	Logical. If TRUE, annotate noise band from <code>noiseBand_low_nN<useCurve></code> and <code>noiseBand_high_nN<useCurve></code> , and add "Noise band" labels above and below the band boundaries.
<code>annotate_repulsive_energy</code>	Logical. If TRUE, add repulsive energy value to subtitle when available.
<code>annotate_adhesive_energy</code>	Logical. If TRUE, add adhesive energy value to subtitle when available.
<code>annotate_adhesive_force</code>	Logical. If TRUE, annotate adhesive force point and separation using arrow + label.
<code>annotate_rupture_distance</code>	Logical. If TRUE, annotate rupture distance/threshold using arrow + label.
<code>annotate_repulsive_distance</code>	Logical. If TRUE, annotate repulsive distance/threshold using arrow + label.
<code>xlim</code>	Optional numeric vector of length 2 for x-axis limits.
<code>ylim</code>	Optional numeric vector of length 2 for y-axis limits.
<code>title</code>	Character. Plot title.
<code>base_size</code>	Numeric. Base theme text size.
<code>line_color</code>	Character. Line color for curves.
<code>point_color</code>	Character. Point color for curves.
<code>line_size</code>	Numeric. Line width for curves.
<code>point_size</code>	Numeric. Point size for curves.
<code>line_alpha</code>	Numeric. Line alpha for curves.
<code>point_alpha</code>	Numeric. Point alpha for curves.
<code>annotation_text_size</code>	Numeric. Text size used for annotation labels.
<code>interaction_label_color_rupture</code>	Character. Text/line color for rupture annotation label.
<code>interaction_label_color_repulsive</code>	Character. Text/line color for repulsive annotation label.
<code>interaction_label_fill</code>	Character. Fill color for interaction label box.
<code>...</code>	Additional ggplot2 components to add to the transformed-curve plot (e.g., <code>theme()</code> , <code>scale_*()</code> , <code>coord_*()</code> , <code>labs()</code> , or extra <code>geom_*()</code> layers).

Value

A ggplot2 object, or a cowplot grid object when plot_raw = TRUE.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_all_analytical_metrics(fd_obj, useCurve = "retract")
plot_a_curve_metrics(fd_obj, curve_name = 'F_1.spm-F4E1_ForceCurveIndex_1.spm', useCurve = "retract", plot_ra
```

plot_complex_heatmap *Plot a scaled feature heatmap using ComplexHeatmap*

Description

Takes a data frame as input together with a set of feature columns to display and optional metadata columns to use as annotations, converts the selected features to numeric, removes incomplete rows, scales each feature by row, and arranges the result as a heatmap matrix with features as rows and samples as columns. Optional annotation columns are added above the heatmap when supplied. The output is a ComplexHeatmap object, or the drawn version of that object when draw = TRUE.

Usage

```
plot_complex_heatmap(
  df,
  include_columns,
  annotate_columns = NULL,
  annotation_colors = NULL,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_row_dend = TRUE,
  show_column_dend = TRUE,
  show_row_names = TRUE,
  show_column_names = TRUE,
  heatmap_name = "z-score",
  row_title = "Features",
  column_title = NULL,
  draw = TRUE
)
```

Arguments

df	A data.frame containing feature and metadata columns. Each row is one sample. Typically the metadata slot of fdobj.
include_columns	Character vector of feature column names to include as heatmap rows.
annotate_columns	Optional character vector (length 0 to 2) of metadata column names used to annotate heatmap columns (samples).

annotation_colors	Optional named list of color mappings for annotation columns passed to ComplexHeatmap::Heatmap(= ...).
cluster_rows	Logical; whether to cluster heatmap rows.
cluster_columns	Logical; whether to cluster heatmap columns.
show_row_dend	Logical; whether to display the row dendrogram.
show_column_dend	Logical; whether to display the column dendrogram.
show_row_names	Logical; whether to display row names (feature names).
show_column_names	Logical; whether to display column names (sample names).
heatmap_name	Character legend title for the scaled values.
row_title	Optional row title.
column_title	Optional column title.
draw	Logical; if TRUE, draws the heatmap immediately.

Value

A ComplexHeatmap object. When draw = TRUE, returns the object produced by ComplexHeatmap::draw().

Examples

```

folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_all_analytical_metrics(fd_obj, useCurve = "retract")
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"
md$surface <- surface
fd_obj@metadata <- md
plot_complex_heatmap(fd_obj@metadata, include_columns = c("adhesive_force_nN_retract", "repulsive_energy_aJ"))

```

plot_deflection_curves

Plot raw deflection vs piezo distance from rawCurves slot of fdObj

Description

Takes an fdObj containing raw AFM curves in fdobj@rawCurves, extracts the requested approach branch, retract branch, or both, and builds a plotting data frame from raw piezo-distance values and raw deflection signals. Optional metadata columns can be used to color curves or split the display into facets. The output is a ggplot2 object showing raw deflection against distance for all included curves.

Usage

```
plot_deflection_curves(
  fdobj,
  curve = c("both", "approach", "retract"),
  group_curves_by = NULL,
  split_curves_by = NULL,
  point_size = 0.5,
  alpha = 0.5,
  line_alpha = 0.5,
  color_map = NULL
)
```

Arguments

fdobj	A 'fdObj' object
curve	Which segment(s) to plot: "approach", "retract", or "both"
group_curves_by	Metadata column to color by (optional)
split_curves_by	Metadata column to facet by (optional)
point_size	Size of points (default = 0.5)
alpha	Transparency of points (default = 0.5)
line_alpha	Transparency of connecting paths (default = 0.5)
color_map	Named vector of colors (optional)

Value

A ggplot2 object of the scatter plots showing raw deflection signal.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
plot_deflection_curves(fd_obj, curve = "retract")
```

plot_deflection_curves_by_index

Plot raw deflection vs row index from rawCurves slot of fdObj

Description

Takes an fdObj with raw curves as input and returns a ggplot2 object showing raw deflection values against data-point index rather than against distance. The function is similar to plot_deflection_curves(), but it uses row number on the x-axis and, when available, respects metadata columns that record the imported number of valid approach or retract points. This makes it useful for checking curve length, signal orientation, and general raw-data shape before downstream transformation.

Usage

```
plot_deflection_curves_by_index(
  fdobj,
  curve = c("both", "approach", "retract"),
  group_curves_by = NULL,
  split_curves_by = NULL,
  point_size = 0.5,
  alpha = 0.5,
  line_alpha = 0.5,
  color_map = NULL
)
```

Arguments

<code>fdobj</code>	A 'fdObj' object
<code>curve</code>	Which segment(s) to plot: "approach", "retract", or "both"
<code>group_curves_by</code>	Metadata column to color by (optional)
<code>split_curves_by</code>	Metadata column to facet by (optional)
<code>point_size</code>	Size of points (default = 0.5)
<code>alpha</code>	Transparency of points (default = 0.5)
<code>line_alpha</code>	Transparency of connecting paths (default = 0.5)
<code>color_map</code>	Named vector of colors (optional)

Details

This is useful for visualizing:

- How many data points are in each curve
- Whether the data is oriented in the correct direction
- The general shape and quality of raw deflection data

Value

A ggplot2 object showing raw deflection signal vs data point index.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
plot_deflection_curves_by_index(fd_obj, curve = "retract")
```

plot_fd_curves

*Plot Force-Distance Curves from an fdObj***Description**

Takes an `fdObj` with transformed approach and/or retract curves as input, extracts the selected transformed curves, and combines them into one plotting table with separation distance on the x-axis and force on the y-axis. Optional metadata columns can be used to color curves or split them across facets, and optional axis limits or extra `ggplot2` layers can be applied. The output is a `ggplot2` object showing force-distance relationships for the requested curves.

Usage

```
plot_fd_curves(
  fdobj,
  curve = "both",
  group_curves_by = NULL,
  split_curves_by = NULL,
  color_map = NULL,
  point_size = 0.5,
  point_alpha = 0.6,
  line_alpha = 0.5,
  xlim = NULL,
  ylim = NULL,
  ...
)
```

Arguments

<code>fdobj</code>	An object of class <code>fdObj</code> .
<code>curve</code>	Character. Either "approach", "retract", or "both" (default = "both").
<code>group_curves_by</code>	Metadata column to color by (optional).
<code>split_curves_by</code>	Metadata column to facet by (optional).
<code>color_map</code>	Named vector of colors (optional).
<code>point_size</code>	Size of points (default = 0.5).
<code>point_alpha</code>	Transparency of points (default = 0.6).
<code>line_alpha</code>	Transparency of curve lines (default = 0.5).
<code>xlim</code>	Optional numeric vector of length 2 for x-axis limits.
<code>ylim</code>	Optional numeric vector of length 2 for y-axis limits.
<code>...</code>	Additional <code>ggplot2</code> layers/settings to add to the plot (e.g., <code>theme(...)</code> , <code>scale_*()</code> , <code>labs(...)</code>).

Value

A `ggplot2` object showing the force-distance relationships from AFM curves.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
plot_fd_curves(fd_obj, curve = "retract")
```

plot_metric_violin	<i>Plot one metric as a violin plot</i>
--------------------	---

Description

Takes a regular data frame as input together with the name of one numeric metric column and one grouping column, then constructs a violin plot for that metric across groups. Optional inputs can add color grouping, faceting, jittered points, boxplots, a linear trend line, and statistical summaries such as a global ANOVA/Kruskal-Wallis test or pairwise t-test/Wilcoxon comparisons. The output is a ggplot object that summarizes the distribution of one metric across user-defined groups.

Usage

```
plot_metric_violin(
  df,
  metric_name,
  group_by,
  color_by = NULL,
  color_map = NULL,
  facet_by = NULL,
  global_test = c("none", "anova", "kruskal"),
  pairwise_test = c("none", "t.test", "wilcox"),
  pairwise_comparisons = NULL,
  p_adjust_method = "BH",
  log10 = FALSE,
  add_points = TRUE,
  show_whisker_box = FALSE,
  add_smooth = FALSE,
  point_alpha = 0.6,
  point_color = "black",
  point_size = 1.2,
  jitter_width = 0.15,
  violin_fill = "grey85",
  violin_color = "grey40",
  violin_alpha = 0.8,
  smooth_color = "steelblue",
  base_size = 10
)
```

Arguments

df	A data.frame containing grouping and metric columns. Typically the metadata slot of fdobj.
metric_name	Character scalar. One column name in df to plot on the y-axis.

group_by	Character scalar. Column name in df used for the x-axis grouping.
color_by	Character scalar. Column name in df used to color violins and points. Defaults to group_by when NULL.
color_map	Optional named character vector of colors for groups in color_by (e.g., c(A = "#1b9e77", B = "#d95f02")).
facet_by	Character vector of length 0 to 2 specifying facet columns. Use NULL for no faceting.
global_test	Character scalar. One of "none", "anova", or "kruskal". When not "none", the global test p-value is added as subtitle.
pairwise_test	Character scalar. One of "none", "t.test", or "wilcox". Pairwise comparisons are shown as significance asterisks.
pairwise_comparisons	Optional list of group-name pairs (e.g., list(c("A", "B"), c("A", "C"))). If NULL, all pairwise combinations are used.
p_adjust_method	Character p-value adjustment method passed to ggpubr::stat_compare_means().
log10	Logical. If TRUE, applies log10 transform to the metric values before plotting and testing. Non-positive values are removed.
add_points	Logical. If TRUE, overlays jittered points.
show_whisker_box	Logical. If TRUE, overlays a whisker boxplot on top of each violin.
add_smooth	Logical. If TRUE, overlays geom_smooth(method = "lm") within each facet.
point_alpha	Numeric point transparency.
point_color	Character point color.
point_size	Numeric point size.
jitter_width	Numeric horizontal jitter width.
violin_fill	Character fill color for violins.
violin_color	Character outline color for violins.
violin_alpha	Numeric violin transparency.
smooth_color	Character color for linear trend line.
base_size	Numeric base font size for theme.

Value

A ggplot object.

Examples

```

folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_all_analytical_metrics(fd_obj, useCurve = "retract")
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"

```

```
md$surface <- surface
fd_obj@metadata <- md
plot_metric_violin(df = fd_obj@metadata, metric_name = "adhesive_force_nN_retract", group_by = "surface", globa
```

plot_pca_biplot

PCA biplot for selected feature columns

Description

Takes a data frame as input together with a set of feature columns and one grouping column used for point color, coerces the selected features to numeric, filters incomplete rows, and runs principal component analysis on the resulting matrix. It then returns a ggplot biplot in which rows of the input data appear as points in PC space and selected feature loadings are drawn as arrows. In addition to the plotted object, the fitted PCA model and the score/loading tables are attached as attributes to the returned plot.

Usage

```
plot_pca_biplot(
  df,
  include_columns,
  color_by,
  color_map = NULL,
  point_size = 2.2,
  point_alpha = 0.9,
  arrow_color = "grey30",
  arrow_alpha = 0.85,
  arrow_scale = 1,
  show_feature_labels = TRUE,
  feature_label_size = 3.5,
  base_size = 12
)
```

Arguments

df	A data.frame containing features and metadata columns. Typically the metadata slot of fdobj.
include_columns	Character vector of column names to include as PCA features.
color_by	Character scalar column name used to color sample points.
color_map	Optional named character vector for sample colors.
point_size	Numeric point size.
point_alpha	Numeric point alpha.
arrow_color	Character arrow/label color for feature loadings.
arrow_alpha	Numeric alpha for loading arrows.
arrow_scale	Numeric multiplier to scale loading-arrow length.
show_feature_labels	Logical; if TRUE, labels loading arrows.

feature_label_size Numeric feature-label text size.

base_size Numeric base font size.

Value

A ggplot object. The fitted PCA model is attached as `\codeattr(plot, "pca_model")`, plus score/loading tables in `\codeattr(plot, "pca_scores")` and `\codeattr(plot, "pca_loadings")`.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "retract", threads = 1, least_length= 300)
fd_obj <- analyze_curves_all_analytical_metrics(fd_obj, useCurve = "retract")
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"
md$surface <- surface
fd_obj@metadata <- md
plot_pca_biplot(fd_obj@metadata, include_columns = c("adhesive_force_nN_retract", "repulsive_energy_aJ_retract"))
```

plot_raw_deflection_heatmap

Heatmap of raw deflection values across all curves in an fdObj

Description

Takes an fdObj containing raw AFM curves as input and converts all raw approach and retract deflection traces into one heatmap-ready matrix. In the resulting matrix, rows correspond to measurement index, columns correspond to individual sample-segment pairs, and raw deflection values are padded and normalized so curves of unequal length can still be displayed together. Optional metadata annotations can be shown above the columns. The output is a ComplexHeatmap object, or a drawn heatmap when draw = TRUE.

Usage

```
plot_raw_deflection_heatmap(
  fdobj,
  annotate_columns = NULL,
  annotation_colors = list(),
  approach_color = "steelblue",
  retract_color = "tomato",
  show_row_names = FALSE,
  show_column_names = FALSE,
  heatmap_name = "Max-min Scaled Deflection",
  col_fun = NULL,
  na_col = "grey90",
```

```

    index_tick_interval = 100,
    draw = TRUE,
    ...
)

```

Arguments

<code>fdobj</code>	An object of class <code>fdObj</code> .
<code>annotate_columns</code>	Character vector of up to 2 metadata column names to display as column annotations alongside the Segment bar. Defaults to <code>NULL</code> .
<code>annotation_colors</code>	Named list of colour vectors for annotations, e.g. <code>list(surface = c(groupA = "green", groupB = "orange"))</code> .
<code>approach_color</code>	Colour for the Approach segment annotation bar. Default <code>"steelblue"</code> .
<code>retract_color</code>	Colour for the Retract segment annotation bar. Default <code>"tomato"</code> .
<code>show_row_names</code>	Logical; show row index labels. Default <code>FALSE</code> .
<code>show_column_names</code>	Logical; show column (sample) names. Default <code>FALSE</code> .
<code>heatmap_name</code>	Character; label for the colour-key legend.
<code>col_fun</code>	Optional <code>circlize::colorRamp2</code> colour function. When <code>NULL</code> a blue–white–red ramp scaled to the 1st/50th/99th percentile of finite values is used.
<code>na_col</code>	Colour for padded NA cells. Default <code>"grey90"</code> .
<code>index_tick_interval</code>	Numeric; spacing between row-index labels added on the right side of the heatmap. Use <code>NULL</code> to disable these labels.
<code>draw</code>	Logical; if <code>TRUE</code> (default), calls <code>ComplexHeatmap::draw()</code> before returning.
<code>...</code>	Additional arguments forwarded to <code>ComplexHeatmap::Heatmap()</code> .

Value

A `ComplexHeatmap` object (drawn when `draw = TRUE`).

Examples

```

folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
md <- fd_obj@metadata
surface <- sub("_.*", "", md$filename)
surface[surface == "F"] <- "Group_A"
surface[surface == "P"] <- "Group_B"
surface[surface == "silicon"] <- "Group_C"
surface[surface == "Z"] <- "Group_D"
md$surface <- surface
fd_obj@metadata <- md
plot_raw_deflection_heatmap(fd_obj, annotate_columns = "surface")

```

run_curvana_app	<i>Launch the Curvana Shiny app</i>
-----------------	-------------------------------------

Description

Starts the interactive Curvana Shiny application bundled with the package.

Usage

```
run_curvana_app(...)
```

Arguments

... Additional arguments passed to shiny::runApp().

Value

Invisibly returns the value from shiny::runApp().

Examples

```
## Not run:  
run_curvana_app()  
  
## End(Not run)
```

show, fdObj-method	<i>Display a Summary of an fdObj Object</i>
--------------------	---

Description

The ‘show()’ method provides a concise overview of an fdObj object. It is automatically called when an fdObj is printed in the console (e.g., typing its name). The summary includes the number of registered samples, imported raw curves, successfully transformed approach/retract curves, and metadata column names.

Usage

```
## S4 method for signature 'fdObj'  
show(object)
```

Arguments

object An fdObj object.

Details

This method is defined for the S4 class fdObj and is executed automatically when the object is shown in the console. It provides a quick overview of data completeness and analysis progress without modifying the object.

Value

Prints a formatted summary of the object in the console.

Examples

```
## Not run:
folder <- system.file("extdata", package = "curvana")
test <- createFdObjFromFolder(folder)
test # calling 'show()' implicitly

## End(Not run)
```

transform_a_curve	<i>Transform a Single AFM Curve into Separation Distance and Force</i>
-------------------	--

Description

Converts one raw AFM force-distance trace from piezo position and deflection signal into a transformed curve containing separation distance and force. The function first subtracts the supplied baseline from the raw deflection signal, converts the corrected deflection from volts to deflection length, multiplies by the spring constant to obtain force in nN, and then estimates a contact-position offset from the supplied sensitivity-calibration segment. That offset is used to shift the piezo axis and compute separation distance.

Usage

```
transform_a_curve(
  x,
  y,
  baseline,
  sensitivity,
  spring_constant,
  senscal_seg_x,
  senscal_seg_y,
  soft = FALSE,
  probe_sensitivity = NULL
)
```

Arguments

x	Numeric vector. Piezo position (distance) in nm.
y	Numeric vector. Deflection signal in volts.
baseline	Numeric. The deflection baseline (in volts).
sensitivity	Numeric. Sensitivity in V/nm.
spring_constant	Numeric. Spring constant in N/m.
senscal_seg_x	Numeric vector. X-values of the sensitivity calibration segment.
senscal_seg_y	Numeric vector. Y-values of the sensitivity calibration segment (in volts).

soft Logical. If TRUE, use `probe_sensitivity` for force conversion while retaining the current transformation equations.

probe_sensitivity Numeric. Probe sensitivity measured on a hard reference surface, required when `soft = TRUE`.

Details

In the default hard-reference workflow, deflection is converted to length using sensitivity. When `soft = TRUE`, force conversion instead uses `probe_sensitivity`, which should come from a hard reference measurement of the same probe. The contact-point estimate and separation-axis calculation still use sensitivity together with the supplied sensitivity-calibration segment.

The contact position is estimated by fitting a linear model to `senscal_seg_x` against baseline-corrected, sensitivity-normalized calibration deflection, and using the fitted intercept as the piezo position corresponding to zero deflection. If baseline, sensitivity, or the calibration segment is missing, the function returns an empty result rather than a partially transformed curve.

Value

A data.frame with:

separation_distance_nm Tip-sample separation (nm).

force_nN Force (nN).

If required inputs are missing, an empty data frame is returned.

Examples

```
#' @examples
x <- c(
  0.000, 0.977, 1.953, 2.930, 3.906, 4.883, 5.859, 6.836, 7.812, 8.789,
  9.766, 10.742, 11.719, 12.695, 13.672, 14.648, 15.625, 16.602, 17.578, 18.555,
  19.531, 20.508, 21.484, 22.461, 23.438, 24.414, 25.391, 26.367, 27.344, 28.320,
  29.297, 30.273, 31.250, 32.227, 33.203, 34.180, 35.156, 36.133, 37.109, 38.086,
  39.062, 40.039, 41.016, 41.992, 42.969, 43.945, 44.922, 45.898, 46.875, 47.852,
  48.828, 49.805, 50.781, 51.758, 52.734, 53.711, 54.688, 55.664, 56.641, 57.617,
  58.594, 59.570, 60.547, 61.523, 62.500, 63.477, 64.453, 65.430, 66.406, 67.383,
  68.359, 69.336, 70.312, 71.289, 72.266, 73.242, 74.219, 75.195, 76.172, 77.148,
  78.125, 79.102, 80.078, 81.055, 82.031, 83.008, 83.984, 84.961, 85.938, 86.914,
  87.891, 88.867, 89.844, 90.820, 91.797, 92.773, 93.750, 94.727, 95.703, 96.680
)

y <- c(
  0.255, 0.243, 0.231, 0.220, 0.208, 0.196, 0.184, 0.172, 0.160, 0.149,
  0.137, 0.125, 0.114, 0.101, 0.088, 0.074, 0.061, 0.048, 0.035, 0.024,
  0.012, 0.001, -0.012, -0.025, -0.037, -0.050, -0.061, -0.070, -0.078, -0.087,
  -0.097, -0.105, -0.109, -0.098, -0.085, -0.072, -0.071, -0.070, -0.066, -0.061,
  -0.057, -0.055, -0.054, -0.055, -0.054, -0.054, -0.054, -0.055, -0.055, -0.055,
  -0.056, -0.056, -0.056, -0.056, -0.055, -0.056, -0.056, -0.056, -0.056, -0.056,
  -0.056, -0.056, -0.056, -0.056, -0.057, -0.057, -0.056, -0.056, -0.055, -0.055,
  -0.055, -0.055, -0.055, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
  -0.056, -0.055, -0.056, -0.055, -0.056, -0.056, -0.056, -0.055, -0.055, -0.056,
  -0.056, -0.055, -0.055, -0.053, -0.054, -0.054, -0.055, -0.056, -0.056, -0.057
)

transformed_df <- transform_a_curve(x, y,
```

```

baseline = -0.05,
sensitivity = 0.012,
spring_constant = 0.1,
senscal_seg_x = x[1:10],
senscal_seg_y = y[1:10]
)

```

transform_curves

Transform All Curves in an fdObj into Separation Distance and Force

Description

Applies transform_a_curve() to every raw curve stored in an fdObj and writes the transformed force-separation curves back into the object. The function operates on either the approach or retract branch, selects the corresponding raw piezo and deflection columns, optionally denoises those raw curves first, and then resolves all quantities required for transformation on a per-curve basis.

Usage

```

transform_curves(
  fdObj,
  spring_constant,
  useCurve = c("approach", "retract"),
  threads = 1,
  denoise_first = FALSE,
  p = 1,
  n = 3,
  m = 0,
  ts = 1,
  end = 200,
  intv = 4,
  R_squared_min = 0.9,
  minimum_length = 4,
  least_length = 150,
  slp_threshold = 0.01,
  std_threshold = 0.05,
  soft = FALSE,
  probe_sensitivity_external = NULL
)

```

Arguments

fdObj	An object of class fdObj.
spring_constant	Numeric or character. If numeric, a constant spring constant (nN/nm) applied to all curves and written to fdObj@metadata\$spring_constant. If character, the name of a column in fdObj@metadata that contains per-curve spring constants.
useCurve	Character; one of c("approach", "retract").
threads	Integer. Number of parallel threads to use (default = 1).
denoise_first	Logical. If TRUE, denoise raw curves for the selected useCurve before transformation.

p	Integer. Savitzky-Golay polynomial order used by <code>denoise_curves()</code> when <code>denoise_first = TRUE</code> .
n	Integer. Savitzky-Golay window size (odd integer) used by <code>denoise_curves()</code> when <code>denoise_first = TRUE</code> .
m	Integer. Savitzky-Golay derivative order used by <code>denoise_curves()</code> when <code>denoise_first = TRUE</code> .
ts	Numeric. Sampling interval used by <code>denoise_curves()</code> when <code>denoise_first = TRUE</code> .
end	Integer. Maximum index considered during sensitivity estimation in <code>analyze_sensitivity()</code> .
intv	Integer. Number of data points added per iteration when building the linear sensitivity segment in <code>calc_sensitivity()</code> via <code>analyze_sensitivity()</code> .
R_squared_min	Numeric. Minimum R^2 threshold for accepting a segment as sufficiently linear during sensitivity calculation in <code>calc_sensitivity()</code> . Default is 0.9.
minimum_length	Integer. Minimum number of accumulated points required for a valid sensitivity result. If the segment is shorter, sensitivity is reported as NA in <code>analyze_sensitivity()</code> .
least_length	Either a single integer (minimum baseline span) or "automatic". Passed to <code>analyze_baseline()</code> when baseline metadata is missing. when "automatic", per-curve span values are read from <code>fdObj@metadata\$baseline_span_approach</code> or <code>fdObj@metadata\$baseline_span_retract</code> depending on <code>useCurve</code> .
slp_threshold	Numeric. Maximum absolute baseline slope allowed in <code>analyze_baseline()</code> .
std_threshold	Numeric. Maximum baseline slope standard error allowed in <code>analyze_baseline()</code> .
soft	Logical. If TRUE, use <code>probe_sensitivity_external</code> for force conversion in <code>transform_a_curve()</code> and pass the resulting sensitivity column to <code>analyze_baseline()</code> . Default FALSE.
probe_sensitivity_external	Numeric or character. Required when <code>soft = TRUE</code> . If numeric, a single probe sensitivity value applied to all curves. If character, the name of an existing column in <code>fdObj@metadata</code> containing per-curve probe sensitivity values. The values are stored in <code>probe_true_sensitivity_V_nm_<useCurve></code> .

Details

Before transforming curves, the function ensures that spring constants, sensitivity values, baseline values, and sensitivity-calibration segments are available. A single numeric `spring_constant` is copied to all curves, whereas a character value is interpreted as the name of a metadata column containing per-curve spring constants. If the required `sensitivity_V_nm_<useCurve>` or `baseline_V_<useCurve>` metadata columns are missing, the function automatically calls `analyze_sensitivity()` and `analyze_baseline()` using the supplied transformation arguments.

When `soft = TRUE`, the function also resolves probe sensitivity values from `probe_sensitivity_external`, stores them in `probe_true_sensitivity_V_nm_<useCurve>`, and passes that information to both baseline analysis and per-curve transformation. If metadata includes the imported number of valid approach or retract points, each raw curve is first truncated to that valid length before transformation so that trailing padding values are ignored.

Curves are transformed independently. For each curve with complete required inputs, `transform_a_curve()` returns separation distance and force. If a curve is missing the required raw columns, has missing baseline, sensitivity, spring constant, or calibration segment, it is not dropped but stored as an empty transformed data frame. Successful results are written to `fdObj@approachCurves` or `fdObj@retractCurves`, depending on `useCurve`, and a summary message reports how many curves failed to transform.

Value

An updated fdObj with transformed curves stored in the corresponding slot.

Examples

```
folder <- system.file("extdata", package = "curvana")
fd_obj <- createFdObjFromFolder(folder)
fd_obj <- transform_curves(fd_obj, spring_constant = 0.1, useCurve = "approach", threads = 1, denoise_first = TRUE)
```

Index

[analyze_a_curve_adhesive_force](#), [2](#)
[analyze_a_curve_area](#), [4](#)
[analyze_a_curve_interaction_distance](#),
[6](#)
[analyze_a_curve_noise](#), [9](#)
[analyze_baseline](#), [11](#)
[analyze_curves_adhesive_force](#), [13](#)
[analyze_curves_all_analytical_metrics](#),
[14](#)
[analyze_curves_energy](#), [17](#)
[analyze_curves_interaction_distance](#),
[18](#)
[analyze_curves_noise](#), [20](#)
[analyze_sensitivity](#), [21](#)
[area_trapezoid](#), [22](#)
[area_triangle](#), [23](#)

[calc_sensitivity](#), [23](#)
[combineFdObj](#), [25](#)
[createFdObjFromFolder](#), [25](#)
[createFdObjFromJPKFolder](#), [27](#)
[crossing_x0](#), [29](#)

[denoise_a_curve](#), [29](#)
[denoise_curves](#), [31](#)

[extract](#), [32](#)

[fdObj-class](#), [32](#)
[find_baseline](#), [33](#)

[plot_a_curve_metrics](#), [37](#)
[plot_all_curve_metrics](#), [35](#)
[plot_complex_heatmap](#), [39](#)
[plot_deflection_curves](#), [40](#)
[plot_deflection_curves_by_index](#), [41](#)
[plot_fd_curves](#), [43](#)
[plot_metric_violin](#), [44](#)
[plot_pca_biplot](#), [46](#)
[plot_raw_deflection_heatmap](#), [47](#)

[run_curvana_app](#), [49](#)

[show, fdObj-method](#), [49](#)

[transform_a_curve](#), [50](#)
[transform_curves](#), [52](#)